Windows Security Band 3: Analyse

Windows-Sicherheitsanalysen durchführen und autorisierte Penetrationstests planen

Von Nmap und Sysinternals über Kerberos-Angriffe und Metasploit bis zu Forensik und Berichten.
Mit 40 Laborübungen im Fünf-Maschinen-Labor, Kontrollfragen und Lernzielkontrollen je Kapitel.

Inhaltsverzeichnis

Vorwort 9

Die Laborumgebung dieses Bandes 9

CLIENT02 und KALI01 einrichten 10

Aufbau dieses Bandes 10

Kapitel 1: Grundlagen des Windows Ethical Hacking 11

1. Einführung 11
2. Technische Grundlagen 11
3. Fachbegriffe 11
4. Praxis und Konfiguration 11
5. Sicherheitsbetrachtung 11
6. Praktische Laborübungen 12
7. Typische Fehler 12
8. Zusammenfassung 12
9. Kontrollfragen 12
10. Lernzielkontrolle 13

Kapitel 2: Windows-Angriffsflächen 14

1. Einführung 14
2. Technische Grundlagen 14
3. Fachbegriffe 14
4. Praxis und Konfiguration 14
5. Sicherheitsbetrachtung 14
6. Praktische Laborübungen 15
7. Typische Fehler 15
8. Zusammenfassung 15
9. Kontrollfragen 15
10. Lernzielkontrolle 16

Kapitel 3: Informationsbeschaffung und Enumeration 17

1. Einführung 17
2. Technische Grundlagen 17
3. Fachbegriffe 17
4. Praxis und Konfiguration 17
5. Sicherheitsbetrachtung 17

6. Praktische Laborübungen . . . 18
7. Typische Fehler . . . 18
8. Zusammenfassung . . . 18
9. Kontrollfragen . . . 19
10. Lernzielkontrolle . . . 19

Kapitel 4: Netzwerkaufklärung mit Nmap . . . 20

1. Einführung . . . 20
2. Technische Grundlagen . . . 20
3. Fachbegriffe . . . 20
4. Praxis und Konfiguration . . . 20
5. Sicherheitsbetrachtung . . . 20
6. Praktische Laborübungen . . . 21
7. Typische Fehler . . . 21
8. Zusammenfassung . . . 21
9. Kontrollfragen . . . 21
10. Lernzielkontrolle . . . 22

Kapitel 5: Netzwerkverkehr mit Wireshark . . . 23

1. Einführung . . . 23
2. Technische Grundlagen . . . 23
3. Fachbegriffe . . . 23
4. Praxis und Konfiguration . . . 23
5. Sicherheitsbetrachtung . . . 23
6. Praktische Laborübungen . . . 24
7. Typische Fehler . . . 24
8. Zusammenfassung . . . 24
9. Kontrollfragen . . . 25
10. Lernzielkontrolle . . . 25

Kapitel 6: Windows-Sicherheitsanalyse mit Sysinternals . . . 26

1. Einführung . . . 26
2. Technische Grundlagen . . . 26
3. Fachbegriffe . . . 26
4. Praxis und Konfiguration . . . 26
5. Sicherheitsbetrachtung . . . 26
6. Praktische Laborübungen . . . 27
7. Typische Fehler . . . 27

8. Zusammenfassung . . . 27
9. Kontrollfragen . . . 28
10. Lernzielkontrolle . . . 28

Kapitel 7: Windows Privilege Escalation . . . 29

1. Einführung . . . 29
2. Technische Grundlagen . . . 29
3. Fachbegriffe . . . 29
4. Praxis und Konfiguration . . . 29
5. Sicherheitsbetrachtung . . . 30
6. Praktische Laborübungen . . . 30
7. Typische Fehler . . . 30
8. Zusammenfassung . . . 30
9. Kontrollfragen . . . 31
10. Lernzielkontrolle . . . 31

Kapitel 8: Active Directory Security Assessment . . . 32

1. Einführung . . . 32
2. Technische Grundlagen . . . 32
3. Fachbegriffe . . . 32
4. Praxis und Konfiguration . . . 32
5. Sicherheitsbetrachtung . . . 32
6. Praktische Laborübungen . . . 33
7. Typische Fehler . . . 33
8. Zusammenfassung . . . 33
9. Kontrollfragen . . . 34
10. Lernzielkontrolle . . . 34

Kapitel 9: Kerberos-Angriffe und Schutzmaßnahmen . . . 35

1. Einführung . . . 35
2. Technische Grundlagen . . . 35
3. Fachbegriffe . . . 35
4. Praxis und Konfiguration . . . 35
5. Sicherheitsbetrachtung . . . 36
6. Praktische Laborübungen . . . 36
7. Typische Fehler . . . 37
8. Zusammenfassung . . . 37
9. Kontrollfragen . . . 37

10. Lernzielkontrolle . . . 37

Kapitel 10: NTLM-Sicherheitsrisiken . . . 38

1. Einführung . . . 38
2. Technische Grundlagen . . . 38
3. Fachbegriffe . . . 38
4. Praxis und Konfiguration . . . 38
5. Sicherheitsbetrachtung . . . 39
6. Praktische Laborübungen . . . 39
7. Typische Fehler . . . 39
8. Zusammenfassung . . . 39
9. Kontrollfragen . . . 40
10. Lernzielkontrolle . . . 40

Kapitel 11: Passwortsicherheit und Hashverfahren . . . 41

1. Einführung . . . 41
2. Technische Grundlagen . . . 41
3. Fachbegriffe . . . 41
4. Praxis und Konfiguration . . . 41
5. Sicherheitsbetrachtung . . . 42
6. Praktische Laborübungen . . . 42
7. Typische Fehler . . . 42
8. Zusammenfassung . . . 42
9. Kontrollfragen . . . 43
10. Lernzielkontrolle . . . 43

Kapitel 12: Windows-Schwachstellenanalyse . . . 44

1. Einführung . . . 44
2. Technische Grundlagen . . . 44
3. Fachbegriffe . . . 44
4. Praxis und Konfiguration . . . 44
5. Sicherheitsbetrachtung . . . 44
6. Praktische Laborübungen . . . 45
7. Typische Fehler . . . 45
8. Zusammenfassung . . . 45
9. Kontrollfragen . . . 45
10. Lernzielkontrolle . . . 46

Kapitel 13: Grundlagen von Metasploit . . . 47

1. Einführung . . . 47
2. Technische Grundlagen . . . 47
3. Fachbegriffe . . . 47
4. Praxis und Konfiguration . . . 47
5. Sicherheitsbetrachtung . . . 47
6. Praktische Laborübungen . . . 48
7. Typische Fehler . . . 48
8. Zusammenfassung . . . 48
9. Kontrollfragen . . . 48
10. Lernzielkontrolle . . . 49

Kapitel 14: PowerShell Security . . . 50

1. Einführung . . . 50
2. Technische Grundlagen . . . 50
3. Fachbegriffe . . . 50
4. Praxis und Konfiguration . . . 50
5. Sicherheitsbetrachtung . . . 50
6. Praktische Laborübungen . . . 51
7. Typische Fehler . . . 51
8. Zusammenfassung . . . 51
9. Kontrollfragen . . . 52
10. Lernzielkontrolle . . . 52

Kapitel 15: Malware- und Rootkit-Analyse . . . 53

1. Einführung . . . 53
2. Technische Grundlagen . . . 53
3. Fachbegriffe . . . 53
4. Praxis und Konfiguration . . . 53
5. Sicherheitsbetrachtung . . . 53
6. Praktische Laborübungen . . . 54
7. Typische Fehler . . . 54
8. Zusammenfassung . . . 54
9. Kontrollfragen . . . 55
10. Lernzielkontrolle . . . 55

Kapitel 16: Windows Event Forensics . . . 56

1. Einführung . . . 56
2. Technische Grundlagen . . . 56

3. Fachbegriffe . . . 56
4. Praxis und Konfiguration . . . 56
5. Sicherheitsbetrachtung . . . 56
6. Praktische Laborübungen . . . 57
7. Typische Fehler . . . 57
8. Zusammenfassung . . . 57
9. Kontrollfragen . . . 58
10. Lernzielkontrolle . . . 58

Kapitel 17: Incident Response . . . 59

1. Einführung . . . 59
2. Technische Grundlagen . . . 59
3. Fachbegriffe . . . 59
4. Praxis und Konfiguration . . . 59
5. Sicherheitsbetrachtung . . . 60
6. Praktische Laborübungen . . . 60
7. Typische Fehler . . . 60
8. Zusammenfassung . . . 60
9. Kontrollfragen . . . 61
10. Lernzielkontrolle . . . 61

Kapitel 18: Digitale Forensik unter Windows . . . 62

1. Einführung . . . 62
2. Technische Grundlagen . . . 62
3. Fachbegriffe . . . 62
4. Praxis und Konfiguration . . . 62
5. Sicherheitsbetrachtung . . . 63
6. Praktische Laborübungen . . . 63
7. Typische Fehler . . . 63
8. Zusammenfassung . . . 63
9. Kontrollfragen . . . 64
10. Lernzielkontrolle . . . 64

Kapitel 19: Security Auditing . . . 65

1. Einführung . . . 65
2. Technische Grundlagen . . . 65
3. Fachbegriffe . . . 65
4. Praxis und Konfiguration . . . 65

5. Sicherheitsbetrachtung . . . 65
6. Praktische Laborübungen . . . 66
7. Typische Fehler . . . 66
8. Zusammenfassung . . . 66
9. Kontrollfragen . . . 66
10. Lernzielkontrolle . . . 67

Kapitel 20: Professionelle Sicherheitsberichte . . . 68

1. Einführung . . . 68
2. Technische Grundlagen . . . 68
3. Fachbegriffe . . . 68
4. Praxis und Konfiguration . . . 68
5. Sicherheitsbetrachtung . . . 68
6. Praktische Laborübungen . . . 69
7. Typische Fehler . . . 69
8. Zusammenfassung . . . 69
9. Kontrollfragen . . . 70
10. Lernzielkontrolle . . . 70
Abschlussprojekt Band 3: Windows Security Audit . . . 70

Verzeichnis der Befehlsbeispiele . . . 71

Index . . . 73

Vorwort

Band 1 hat einen Windows-Client aufgebaut, Band 2 ihn in eine gehärtete Active-Directory-Domäne eingebunden. Band 3 prüft, ob diese Härtung tatsächlich wirkt, indem es dieselben Methoden anwendet, mit denen auch ein Angreifer vorgehen würde, ausschließlich im eigenen, isolierten Labor. Der Bogen reicht von der Netzwerkerkundung über Kerberos- und NTLM-Angriffe, Passwort-Audits und ein Exploitation-Framework bis zur forensischen Nachbereitung eines Vorfalls und einem professionellen Abschlussbericht.

Dieses Buch setzt Band 1 und 2 als bekannt voraus. Es behandelt teilweise dieselben Werkzeuge wie die Linux-Schwesterreihe (Nmap, Wireshark, Metasploit), hier aber konsequent auf Windows-Umgebungen und Active Directory bezogen.

Die Laborumgebung dieses Bandes

Band 3 vervollständigt das Labor aus dem Master-Prompt um die beiden fehlenden Maschinen:

System	Rolle	Betriebssystem
DC01	Domain Controller aus Band 2	Windows Server 2022
SRV01	Mitgliedsserver aus Band 2, jetzt Prüfobjekt	Windows Server 2022
CLIENT01	Arbeitsplatzrechner aus Band 1/2	Windows 11 Pro
CLIENT02	bewusst einfach konfigurierter Testclient, neu in diesem Band	Windows 11 Pro
KALI01	Analysewerkstatt für autorisierte Sicherheitsprüfungen, neu in diesem Band	Kali Linux

Beispiel V.1: Erweiterte Netzwerkarchitektur

```
DC01       192.168.56.10   (Windows Server 2022, Domain Controller)
SRV01      192.168.56.20   (Windows Server 2022, Mitgliedsserver)
CLIENT01   192.168.56.50   (Windows 11 Pro, Domänenmitglied)
CLIENT02   192.168.56.60   (Windows 11 Pro, bewusst einfach konfiguriert)
KALI01     192.168.56.70   (Kali Linux)
Domäne:    nova.int, alle fünf Systeme im selben, isolierten Netzwerk.
```

WARNUNG – Sicherheitsrisiko

Jede in diesem Buch gezeigte Analyse- oder Angriffstechnik ist ausschließlich für den Einsatz gegen die fünf Systeme dieses isolierten Labors bestimmt, oder im Rahmen eines schriftlich autorisierten, realen Auftrags. Der Einsatz gegen fremde, nicht ausdrücklich freigegebene Systeme ist nach §§ 202a-202c sowie § 303b StGB strafbar, unabhängig vom Ergebnis.

CLIENT02 und KALI01 einrichten

Beispiel V.2: CLIENT02 einrichten

```
1. Windows 11 Pro wie CLIENT01 installieren (Band 1, Beispiel V.2)
2. Der Domäne nova.int beitreten lassen, aber bewusst OHNE die
   Härtungsmaßnahmen aus Band 2 (keine Baseline-GPO-Verknüpfung,
   Standard-Organisationseinheit "Computers" belassen)
3. Ein Testkonto mit schwachem, aber nicht trivialem Passwort anlegen
```

Beispiel V.3: KALI01 einrichten

```
1. Aktuelles Kali-Linux-Image installieren, Netzwerkadapter auf
   dasselbe isolierte Netzwerk wie die übrigen vier Systeme stellen
2. $ ping -c 3 192.168.56.10   # Erreichbarkeit von DC01 prüfen
```

Aufbau dieses Bandes

Zwanzig Kapitel, gegliedert in fünf Abschnitte: Grundlagen und Aufklärung (1-6), Privilege Escalation und AD-Angriffe (7-10), Passwortsicherheit und Exploitation (11-13), PowerShell- und Malware-Analyse (14-15), sowie Forensik, Reaktion und Berichtswesen (16-20).

Kapitel 1: Grundlagen des Windows Ethical Hacking

Lernziele

- Du unterscheidest White-, Grey- und Black-Hat-Hacking nach Autorisierung.
- Du benennst die rechtlichen Grundlagen für autorisierte Sicherheitstests in Deutschland.
- Du formulierst einen Scope für eine Windows-/AD-Umgebung.

1. Einführung

Jede Technik in diesem Buch gleicht technisch dem, was ein Angreifer gegen eine Windows-Domäne einsetzen würde. Der einzige Unterschied ist die Autorisierung. Dieses Kapitel legt fest, was das konkret bedeutet, bevor Kapitel 3 die erste praktische Technik einführt.

2. Technische Grundlagen

Ethical Hacking bezeichnet die autorisierte Anwendung von Angriffstechniken, um Schwachstellen vor einem tatsächlichen Angreifer zu finden. White Hat (autorisiert), Grey Hat (ohne klare Autorisierung, ohne böswillige Absicht) und Black Hat (unautorisiert und böswillig) unterscheiden sich ausschließlich rechtlich und ethisch, nicht technisch.

Beispiel 1.1: Derselbe Angriff, drei rechtliche Einordnungen (Konzept)

```
White Hat:  Kerberoasting-Test gegen DC01, mit Vertrag und definiertem Scope
Grey Hat:   dieselbe Technik gegen ein fremdes System ohne Auftrag, Fund gemeldet
Black Hat:  dieselbe Technik gegen ein fremdes System ohne Auftrag, ausgenutzt
```

3. Fachbegriffe

Ein **Scope** legt vertraglich fest, welche Systeme ein Test umfassen darf. Eine **Freistellungserklärung** bestätigt schriftlich die Autorisierung. §§ 202a-202c StGB bilden den zentralen strafrechtlichen Rahmen in Deutschland, § 303b StGB erfasst zusätzlich Computersabotage.

4. Praxis und Konfiguration

Beispiel 1.2: Mindestinhalt eines AD-spezifischen Scope-Dokuments

```
1. Zielsysteme (DC01, SRV01, CLIENT01, CLIENT02), explizit ausgeschlossene Systeme
2. Zeitfenster des Tests
3. Erlaubte Methoden (z. B. Kerberoasting: ja, Denial-of-Service gegen DC01: nein)
4. Notfallkontakt beim Auftraggeber
5. Vertraulichkeitsregeln für gefundene Zugangsdaten und Hashes
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Es gibt keine Technik in diesem Buch, die „nur für legale Zwecke“ reserviert ist. Prüfe vor jeder Übung in diesem Buch, ob du dich tatsächlich innerhalb des im Vorwort definierten Labors bewegst.

6. Praktische Laborübungen

Übung A: Einen Scope für das eigene Labor formulieren

Aufgabenstellung: Formuliere ein Scope-Dokument nach Beispiel 1.2 für die Übungen dieses Buches. **Ausgangssituation:** Das Fünf-Maschinen-Labor aus dem Vorwort ist eingerichtet. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** Beispiel 1.2 mit den konkreten Angaben aus dem Vorwort ausfüllen. **Erwartetes Ergebnis:** Ein kurzes, vollständiges Dokument mit allen fünf Pflichtangaben. **Überprüfung:** Jeder Punkt ist konkret, nicht nur platzhalterhaft, ausgefüllt. **Fehleranalyse:** Unklare Angabe? Mit den tatsächlichen IP-Adressen aus dem Vorwort abgleichen. **Musterlösung:** Ein vollständiges Scope-Dokument, das alle fünf Systeme als Ziel und das externe Internet als ausgeschlossen benennt.

Übung B: Einordnung realer Szenarien

Aufgabenstellung: Ordne drei kurze, fiktive Szenarien als White, Grey oder Black Hat ein und begründe. **Ausgangssituation:** keine. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** Drei Szenarien formulieren, jeweils nach Autorisierung bewerten. **Erwartetes Ergebnis:** Eine begründete Einordnung je Szenario. **Überprüfung:** Die Begründung bezieht sich auf die Autorisierung, nicht auf die Technik. **Fehleranalyse:** Einordnung nach Technik vorgenommen? Zurück zu Beispiel 1.1. **Musterlösung:** Eine korrekte Lösung erkennt, dass identische Technik je nach Autorisierung alle drei Kategorien ergeben kann.

7. Typische Fehler

Fehler	Ursache	Behebung
Technik statt Autorisierung als Unterscheidungsmerkmal	Verwechslung von Mittel und Rahmen	immer zuerst nach Autorisierung fragen
Kein schriftlicher Scope vor Testbeginn	Bequemlichkeit	Scope immer schriftlich festhalten
Eigenes Labor für „sowieso erlaubt“ gehalten	fehlende Dokumentation	auch im eigenen Labor Vorgehen dokumentieren

8. Zusammenfassung

Zusammenfassung

- White, Grey und Black Hat unterscheiden sich durch Autorisierung, nicht durch Technik.
- §§ 202a-202c und § 303b StGB bilden den zentralen strafrechtlichen Rahmen in Deutschland.
- Ein schriftlicher Scope ist die Voraussetzung jedes autorisierten Tests.
- Dieses Buch arbeitet ausschließlich im isolierten, eigenen Fünf-Maschinen-Labor.

9. Kontrollfragen

1. Was unterscheidet White Hat von Black Hat? (leicht)

2. Welche Paragrafen bilden den zentralen strafrechtlichen Rahmen? (leicht)
3. Was gehört mindestens in ein Scope-Dokument? (mittel)
4. Wozu dient eine Freistellungserklärung? (mittel)
5. Ein Kollege möchte „nur kurz“ ein System außerhalb des vereinbarten Scopes mittesten. Wie reagierst du? (schwer)

Lösungshinweise: 1. Ausschließlich die Autorisierung, nicht die eingesetzte Technik. 2. §§ 202a-202c StGB und § 303b StGB. 3. Zielsysteme, Zeitfenster, erlaubte Methoden, Notfallkontakt, Vertraulichkeitsregeln. 4. Sie belegt im Ernstfall gegenüber Behörden die Autorisierung des Tests. 5. Ablehnen und auf die schriftliche Scope-Erweiterung bestehen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] White, Grey und Black Hat anhand der Autorisierung unterscheiden kannst,
- [ ] ein vollständiges Scope-Dokument für das eigene Labor erstellt hast,
- [ ] die zentralen Paragrafen benennen kannst,
- [ ] auf eine Scope-Überschreitung angemessen reagieren würdest.

Kapitel 2: Windows-Angriffsflächen

Lernziele

- Du erstellst ein STRIDE-Bedrohungsmodell für eine Windows-Domäne.
- Du ordnest typische AD-spezifische Vertrauensgrenzen ein.
- Du leitest aus einem Bedrohungsmodell Testschwerpunkte ab.

1. Einführung

Bevor die praktische Analyse beginnt, braucht es ein systematisches Bild dessen, wonach überhaupt gesucht werden soll – in einer Domäne komplexer als bei einem einzelnen Client, weil Vertrauensbeziehungen zwischen Systemen hinzukommen.

2. Technische Grundlagen

STRIDE kategorisiert Bedrohungen in Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege. In einer AD-Umgebung kommen spezifische Vertrauensgrenzen hinzu: zwischen Client und Domain Controller, zwischen Domänenmitgliedern untereinander, und zwischen unterschiedlich privilegierten Konten.

Beispiel 2.1: Vertrauensgrenzen in der Labor-Domäne (Konzept)

```
CLIENT02 (schwach konfiguriert) <-- Vertrauensgrenze --> SRV01 (Mitgliedsserver)
SRV01 <-- Vertrauensgrenze --> DC01 (Domain Controller, höchster Schutzbedarf)
```

3. Fachbegriffe

Ein **Angriffsvektor** ist der konkrete Weg, über den eine Bedrohung ein System erreicht. **Lateral Movement** bezeichnet die Bewegung eines Angreifers von einem bereits kompromittierten System zu weiteren Systemen innerhalb derselben Umgebung.

4. Praxis und Konfiguration

Beispiel 2.2: STRIDE auf die Labor-Domäne angewendet (Auszug)

```
S - Spoofing: gefälschte Absenderadresse bei SMB-Zugriff auf SRV01
T - Tampering: Veränderung einer GPO durch ein zu breit delegiertes Konto
I - Info Disclosure: Kerberos-Ticket mit schwach gehashtem Dienstkonto-Passwort
D - Denial of Service: Kontosperrung vieler Benutzer durch absichtliche Fehlversuche
E - Elevation of Privilege: Rechteausweitung über eine Fehlkonfiguration auf CLIENT02
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Ein Bedrohungsmodell ist eine Prognose, keine vollständige Garantie. Es lenkt die Analyse auf plausible Schwerpunkte, ersetzt aber nicht die tatsächliche, praktische Prüfung in den folgenden Kapiteln.

6. Praktische Laborübungen

Übung A: Ein Bedrohungsmodell für die Domäne erstellen

Aufgabenstellung: Erstelle ein STRIDE-Modell für die gesamte Labor-Domäne nach Beispiel 2.2. **Ausgangssituation:** Alle fünf Systeme nach Vorwort eingerichtet. **Voraussetzungen:** Kenntnis der Konfiguration aus Band 2. **Schritt-für-Schritt-Anleitung:** Für jede STRIDE-Kategorie mindestens eine plausible, domänenspezifische Bedrohung formulieren. **Erwartetes Ergebnis:** Sechs benannte Bedrohungen. **Überprüfung:** Jede Bedrohung bezieht sich auf ein tatsächlich vorhandenes System oder eine Vertrauensgrenze. **Fehleranalyse:** Bedrohung zu allgemein? Auf eine konkrete, tatsächlich vorhandene Komponente beziehen. **Musterlösung:** Ein vollständiges Modell mit sechs konkreten, auf die Domäne bezogenen Bedrohungen.

Übung B: Testschwerpunkte ableiten

Aufgabenstellung: Leite aus dem Bedrohungsmodell drei konkrete Testschwerpunkte für die folgenden Kapitel ab. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** Die plausibelsten drei Bedrohungen auswählen, jeweils einem späteren Kapitel zuordnen. **Erwartetes Ergebnis:** Eine priorisierte Liste mit Kapitelzuordnung. **Überprüfung:** Jeder Schwerpunkt lässt sich einem konkreten, später behandelten Werkzeug zuordnen. **Fehleranalyse:** Keine passende Zuordnung? Inhaltsverzeichnis dieses Bandes konsultieren. **Musterlösung:** Eine vollständige Lösung verbindet jede Bedrohung mit einem konkreten, später folgenden Kapitel.

7. Typische Fehler

Fehler	Ursache	Behebung
Bedrohungsmodell nur für Clients, nicht für DC01 erstellt	Domain Controller übersehen	alle fünf Systeme gleichwertig einbeziehen
Vertrauensgrenzen zwischen Systemen ignoriert	Domäne als Einzelsystem gedacht	Beziehungen zwischen Systemen explizit modellieren
Lateral Movement nicht mitgedacht	Fokus nur auf Einzelsysteme	Bewegungsmöglichkeiten zwischen Systemen einbeziehen

8. Zusammenfassung

Zusammenfassung

- STRIDE strukturiert Bedrohungen in sechs Kategorien, auch in AD-Umgebungen anwendbar.
- Vertrauensgrenzen zwischen Client, Server und Domain Controller sind domänenspezifisch wichtig.
- Lateral Movement beschreibt die Bewegung eines Angreifers zwischen bereits erreichten Systemen.
- Aus dem Modell lassen sich konkrete Testschwerpunkte für die folgenden Kapitel ableiten.

9. Kontrollfragen

1. Wofür steht STRIDE? (leicht)
2. Was ist Lateral Movement? (leicht)
3. Warum sind Vertrauensgrenzen in einer Domäne komplexer als bei einem Einzelsystem? (mittel)

4. Was ist ein Angriffsvektor? (mittel)
5. Zwei Clients haben identische Software, aber unterschiedliche Vertrauensgrenzen (einer ist Domänenmitglied mit Zugriff auf eine Fileshare, einer nicht). Sollte ihr Bedrohungsmodell identisch sein? (schwer)

Lösungshinweise: 1. Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege. 2. Die Bewegung eines Angreifers von einem bereits kompromittierten System zu weiteren Systemen. 3. Weil mehrere Systeme mit unterschiedlichen Privilegien und Vertrauensbeziehungen zueinander stehen, nicht nur ein isoliertes Gerät. 4. Der konkrete Weg, über den eine Bedrohung ein System erreicht. 5. Nein, der zusätzliche Zugriff auf eine Fileshare eröffnet zusätzliche Angriffsvektoren und sollte im Modell berücksichtigt werden.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] ein STRIDE-Modell für die gesamte Domäne erstellt hast,
- [ ] Vertrauensgrenzen zwischen Systemen benennen kannst,
- [ ] aus einem Modell konkrete Testschwerpunkte ableitest,
- [ ] Lateral Movement als Konzept erklären kannst.

Kapitel 3: Informationsbeschaffung und Enumeration

Lernziele

- Du sammelst grundlegende Domäneninformationen ohne privilegierten Zugang.
- Du nutzt PowerView-ähnliche Techniken zur AD-Enumeration.
- Du identifizierst Benutzer- und Gruppenstrukturen von einem eingeschränkten Konto aus.

1. Einführung

Bevor sich eine Schwachstelle ausnutzen lässt, muss sie gefunden werden. Dieses Kapitel zeigt, wie viel Information bereits ein gewöhnliches, eingeschränktes Domänenkonto über die gesamte Umgebung preisgibt.

2. Technische Grundlagen

Jedes authentifizierte Domänenkonto kann standardmäßig große Teile der Active-Directory-Struktur lesen: Benutzer, Gruppen, Organisationseinheiten, teilweise auch Kontoattribute anderer Benutzer. Diese weitreichende Lesbarkeit ist von Active Directory so vorgesehen, macht aber jedes kompromittierte Konto zu einem wertvollen Ausgangspunkt für einen Angreifer.

Beispiel 3.1: Domäneninformationen mit Bordmitteln sammeln

```
PS> Get-ADDomain
PS> Get-ADUser -Filter * -Properties MemberOf | Select-Object Name, MemberOf
```

3. Fachbegriffe

PowerView ist ein verbreitetes PowerShell-Werkzeug speziell für AD-Enumeration, Teil des größeren PowerSploit-Werkzeugkastens. **BloodHound** visualisiert Berechtigungsbeziehungen in Active Directory als Graph und macht nicht offensichtliche, mehrstufige Angriffspfade sichtbar.

4. Praxis und Konfiguration

Beispiel 3.2: Mitglieder privilegierter Gruppen von einem eingeschränkten Konto aus ermitteln

```
PS> Get-ADGroupMember -Identity "Domain Admins"
```

Beispiel 3.3: Freigaben auf SRV01 ohne besondere Rechte auflisten

```
PS> net view \\SRV01
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Führe Enumeration ausschließlich gegen die Labor-Domäne aus. Bereits das Sammeln von Informationen über ein fremdes Netzwerk ohne Autorisierung kann als Vorbereitungshandlung nach § 202c StGB relevant sein.

Die Tatsache, dass ein gewöhnliches Konto so viel sehen kann, ist kein Fehler, sondern die Grundfunktion von Active Directory – die Konsequenz ist, dass jedes einzelne Konto in der Domäne als potenzieller Ausgangspunkt für einen Angreifer behandelt werden sollte (Band 2, Kapitel 5).

6. Praktische Laborübungen

Übung A: Domänenstruktur von einem eingeschränkten Konto aus kartieren

Aufgabenstellung: Melde dich mit dem schwachen Testkonto auf CLIENT02 an und sammle so viele Domäneninformationen wie möglich. **Ausgangssituation:** CLIENT02 mit Testkonto aus dem Vorwort. **Voraussetzungen:** keine privilegierten Rechte. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 3.1, 3.2. **Erwartetes Ergebnis:** Eine vollständige Liste aller Benutzer, Gruppen und der Mitglieder privilegierter Gruppen. **Überprüfung:** Die gesammelten Informationen stimmen mit der tatsächlichen, aus Band 2 bekannten Struktur überein. **Fehleranalyse:** Zugriff verweigert? Prüfen, ob das Testkonto tatsächlich ein authentifiziertes Domänenkonto ist. **Musterlösung:** Eine vollständige Dokumentation der Domänenstruktur, erreicht ohne jede besondere Berechtigung.

Übung B: Freigaben und Dienste im Netzwerk auflisten

Aufgabenstellung: Liste erreichbare Freigaben auf SRV01 auf und dokumentiere, welche davon für das Testkonto zugänglich sind. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 3.3, danach Zugriff auf jede gefundene Freigabe testen. **Erwartetes Ergebnis:** Eine Liste der Freigaben mit Zugriffsstatus für das Testkonto. **Überprüfung:** Der dokumentierte Zugriffsstatus stimmt mit den tatsächlich konfigurierten Freigabeberechtigungen überein. **Fehleranalyse:** Keine Freigaben gefunden? Prüfen, ob SRV01 tatsächlich Freigaben aus früheren Übungen (Band 2, Kapitel 8) besitzt. **Musterlösung:** Eine vollständige, dokumentierte Zugriffsübersicht.

7. Typische Fehler

Fehler	Ursache	Behebung
Umfang der AD-Lesbarkeit unterschätzt	Annahme, ein Standardkonto sehe „fast nichts“	aktiv mit Beispiel 3.1 nachprüfen
Enumeration außerhalb des Labors versucht	Scope aus Kapitel 1 nicht beachtet	ausschließlich gegen die eigene Domäne arbeiten
BloodHound ungeprüft gegen Produktivsysteme eingesetzt	fehlendes Bewusstsein für die Datenmenge	nur im eigenen, isolierten Labor einsetzen

8. Zusammenfassung

Zusammenfassung

- Jedes authentifizierte Domänenkonto kann standardmäßig große Teile der AD-Struktur lesen.
- PowerView und BloodHound sind verbreitete Werkzeuge zur systematischen AD-Enumeration.
- Freigaben lassen sich auch ohne besondere Rechte häufig auflisten.

> - Jedes Konto in einer Domäne ist ein potenzieller Ausgangspunkt für einen Angreifer.

9. Kontrollfragen

1. Was kann ein gewöhnliches, authentifiziertes Domänenkonto standardmäßig lesen? (leicht)
2. Was visualisiert BloodHound? (leicht)
3. Warum ist diese weitreichende Lesbarkeit kein Fehler, sondern Grundfunktion? (mittel)
4. Welcher Befehl listet Freigaben eines Servers im Netzwerk auf? (mittel)
5. Ein Unternehmen argumentiert, da „nur normale Mitarbeiterkonten“ betroffen wären, sei ein Phishing-Vorfall gegen ein Standardkonto kein kritisches Risiko. Was entgegnest du mit dem Wissen aus diesem Kapitel? (schwer)

Lösungshinweise: 1. Große Teile der AD-Struktur: Benutzer, Gruppen, Organisationseinheiten, teils Kontoattribute. 2. Berechtigungsbeziehungen in Active Directory als Graph, inklusive mehrstufiger Angriffspfade. 3. Weil Active Directory so konzipiert ist, dass authentifizierte Nutzer die Struktur einsehen können müssen, um normal arbeiten zu können. 4. `net view \\<servername>`. 5. Selbst ein „normales“ Konto kann die gesamte Domänenstruktur auslesen und damit wertvolle Ausgangsinformationen für weitere Angriffsschritte liefern; jedes kompromittierte Konto ist ein potenzieller erster Schritt, unabhängig von seinen eigenen Rechten.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] die Domänenstruktur von einem eingeschränkten Konto aus kartiert hast,
- [ ] erreichbare Freigaben dokumentiert hast,
- [ ] erklären kannst, warum jedes Konto ein potenzieller Ausgangspunkt ist,
- [ ] PowerView und BloodHound in ihrem Zweck einordnen kannst.

Kapitel 4: Netzwerkaufklärung mit Nmap

Lernziele

- Du identifizierst Windows-spezifische Dienste über Nmap-Scans.
- Du erkennst einen Domain Controller allein anhand offener Ports.
- Du vergleichst die Angriffsfläche von CLIENT01 (gehärtet) und CLIENT02 (ungehärtet).

1. Einführung

Nmap ist werkzeugübergreifend dieselbe Software wie in der Linux-Schwesterreihe, liefert gegen Windows-Systeme aber eigene, charakteristische Ergebnisse.

2. Technische Grundlagen

Ein Domain Controller lässt sich oft bereits an seiner typischen Portkombination erkennen: 389/636 (LDAP/LDAPS), 88 (Kerberos), 445 (SMB), 53 (DNS) gemeinsam offen sind ein starkes Indiz.

Beispiel 4.1: DC01 identifizieren

```
$ sudo nmap -sS -sV -p 53,88,389,445,636 192.168.56.10
```

3. Fachbegriffe

LDAP (Lightweight Directory Access Protocol) ist das Protokoll, über das Active Directory abgefragt wird. Ein Nmap-NSE-Skript wie `smb-os-discovery` kann teilweise Betriebssystemversion und Domänenzugehörigkeit direkt aus SMB-Antworten auslesen.

4. Praxis und Konfiguration

Beispiel 4.2: SMB-Details mit NSE auslesen

```
$ sudo nmap --script=smb-os-discovery -p 445 192.168.56.20
```

Beispiel 4.3: Vergleichsscan gegen beide Clients

```
$ sudo nmap -sS -sV -p- 192.168.56.50
$ sudo nmap -sS -sV -p- 192.168.56.60
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Scanne ausschließlich die fünf Laborsysteme. Ein Scan gegen einen echten, fremden Domain Controller ist eine aktive, unautorisierte Kontaktaufnahme mit einem besonders kritischen System.

6. Praktische Laborübungen

Übung A: DC01 anhand offener Ports identifizieren

Aufgabenstellung: Führe Beispiel 4.1 aus und bestätige, dass die Portkombination auf einen Domain Controller hindeutet. **Ausgangssituation:** KALI01 mit Netzwerkzugriff auf DC01. **Voraussetzungen:** Nmap auf KALI01. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 4.1. **Erwartetes Ergebnis:** Alle fünf typischen Ports erscheinen offen. **Überprüfung:** Die Kombination entspricht der in Kapitel 2 beschriebenen DC-Signatur. **Fehleranalyse:** Einzelne Ports geschlossen? Firewall-Konfiguration auf DC01 aus Band 2 prüfen. **Musterlösung:** Eine bestätigte, anhand der Portkombination eindeutig identifizierte DC-Rolle.

Übung B: CLIENT01 und CLIENT02 vergleichen

Aufgabenstellung: Führe Beispiel 4.3 gegen beide Clients aus und vergleiche das Ergebnis. **Ausgangssituation:** CLIENT01 (gehärtet, Band 1/2) und CLIENT02 (ungehärtet, Vorwort dieses Bandes). **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 4.3. **Erwartetes Ergebnis:** CLIENT02 zeigt mehr offene Ports und auswertbarere Bannerinformationen als CLIENT01. **Überprüfung:** Die Differenz lässt sich auf konkrete Band-2-Härtungsmaßnahmen zurückführen. **Fehleranalyse:** Kein Unterschied? Prüfen, ob CLIENT02 tatsächlich ohne Härtungs-GPO konfiguriert wurde. **Musterlösung:** Eine Tabelle, die für jeden Port die Differenz zwischen beiden Clients benennt und begründet.

7. Typische Fehler

Fehler	Ursache	Behebung
Scan gegen falsches Ziel	IP-Adresse verwechselt	Adressen aus dem Vorwort vor jedem Scan prüfen
DC-Erkennung nur an einem einzelnen Port festgemacht	unvollständiges Muster	immer die vollständige, typische Portkombination prüfen
Unterschied zwischen Clients nicht erklärt	Band-2-Maßnahmen nicht mehr präsent	Band 2 bei Bedarf gezielt nachschlagen

8. Zusammenfassung

Zusammenfassung

- Eine typische Portkombination (53, 88, 389, 445, 636) identifiziert einen Domain Controller zuverlässig.
- NSE-Skripte wie `smb-os-discovery` liefern zusätzliche Details aus SMB-Antworten.
- Der Vergleich zwischen gehärtetem und ungehärtetem Client macht Band-2-Maßnahmen messbar.
- Jeder Scan gehört ausschließlich gegen die eigenen Laborsysteme.

9. Kontrollfragen

1. Welche Ports deuten typischerweise auf einen Domain Controller hin? (leicht)
2. Wofür steht LDAP? (leicht)
3. Was leistet das NSE-Skript `smb-os-discovery`? (mittel)
4. Warum zeigt CLIENT02 mehr offene Ports als CLIENT01? (mittel)

5. Ein Scan zeigt Port 88 offen, aber nicht 389. Ist das System dennoch wahrscheinlich ein Domain Controller? (schwer)

Lösungshinweise: 1. 53, 88, 389, 445, 636. 2. Lightweight Directory Access Protocol. 3. Es liest Betriebssystemversion und teilweise Domänenzugehörigkeit direkt aus SMB-Antworten aus. 4. Weil CLIENT01 die Härtungsmaßnahmen aus Band 2 (Firewall, deaktivierte Dienste) erhalten hat, CLIENT02 bewusst nicht. 5. Eher unwahrscheinlich als vollständiger Domain Controller; einzelne offene Ports können auch andere Rollen oder Fehlkonfigurationen anzeigen, die vollständige Kombination ist das verlässlichere Indiz.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] DC01 anhand seiner Portkombination identifiziert hast,
- [ ] ein NSE-Skript gezielt gegen SMB eingesetzt hast,
- [ ] die Ergebnisse von CLIENT01 und CLIENT02 begründet verglichen hast,
- [ ] Scan-Ergebnisse dokumentiert gespeichert hast.

Kapitel 5: Netzwerkverkehr mit Wireshark

Lernziele

- Du schneidest Kerberos- und LDAP-Verkehr gezielt mit.
- Du erkennst einen NTLM-Fallback im Mitschnitt.
- Du unterscheidest verschlüsselten von unverschlüsseltem LDAP-Verkehr.

1. Einführung

Nmap (Kapitel 4) zeigt, welche Dienste erreichbar sind. Wireshark zeigt, was tatsächlich über die Leitung geht – besonders aufschlussreich bei Authentifizierungsprotokollen wie Kerberos, NTLM und LDAP.

2. Technische Grundlagen

Kerberos-Verkehr (Band 2, Kapitel 7) lässt sich in Wireshark anhand von AS-REQ, AS-REP, TGS-REQ und TGS-REP-Nachrichten identifizieren. **LDAP** kann unverschlüsselt (Port 389) oder verschlüsselt als **LDAPS** (Port 636) übertragen werden.

Beispiel 5.1: Kerberos-Verkehr filtern

```
Display-Filter: kerberos
```

3. Fachbegriffe

Ein **AS-REQ** ist die erste Anfrage eines Clients an den Domain Controller zur Authentifizierung. Unverschlüsselter LDAP-Verkehr kann bei einer einfachen Bindung (Simple Bind) Zugangsdaten im Klartext übertragen, wenn keine zusätzliche Verschlüsselung (LDAPS oder Signierung) erzwungen wird.

4. Praxis und Konfiguration

Beispiel 5.2: Unverschlüsselten LDAP-Verkehr auf Klartext-Zugangsdaten prüfen

```
Display-Filter: ldap.name contains "bindRequest"
```

Beispiel 5.3: NTLM-Fallback im Mitschnitt erkennen

```
Display-Filter: ntlmssp
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

> Schneide ausschließlich Verkehr in deinem eigenen, isolierten Labor mit. Das unbefugte Abfangen fremder Datenübermittlung ist nach § 202b StGB strafbar.

Findet sich bei einer eigentlich kerberos-fähigen Domänenumgebung dennoch NTLM-Verkehr, deutet das auf eine der in Band 2, Kapitel 7 beschriebenen Fallback-Ursachen hin, etwa DNS-Probleme oder eine Namensauflösung über IP-Adresse statt Hostname.

6. Praktische Laborübungen

Übung A: Kerberos-Anmeldung mitschneiden

Aufgabenstellung: Schneide eine Anmeldung von CLIENT01 an DC01 mit und identifiziere AS-REQ und AS-REP. **Ausgangssituation:** CLIENT01 in der Domäne, Wireshark auf KALI01 oder direkt auf CLIENT01. **Voraussetzungen:** Mitschnitt-Rechte. **Schritt-für-Schritt-Anleitung:** Mitschnitt starten, auf CLIENT01 ab- und erneut anmelden, Beispiel 5.1 anwenden. **Erwartetes Ergebnis:** Mindestens ein AS-REQ/AS-REP-Paar erscheint im gefilterten Mitschnitt. **Überprüfung:** Zeitstempel passt zur tatsächlichen Anmeldung. **Fehleranalyse:** Kein Kerberos-Verkehr sichtbar? Prüfen, ob der Mitschnitt tatsächlich am richtigen Netzwerksegment erfolgt. **Musterlösung:** Ein dokumentierter Kerberos-Anmeldeablauf im Mitschnitt.

Übung B: NTLM-Fallback provozieren und erkennen

Aufgabenstellung: Verbinde dich testweise über die IP-Adresse statt des Hostnamens mit SRV01 und beobachte einen möglichen NTLM-Fallback. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** Mitschnitt starten, Verbindung zu `\\192.168.56.20` statt `\\SRV01` aufbauen, Beispiel 5.3 anwenden. **Erwartetes Ergebnis:** NTLM-Pakete erscheinen, da Kerberos eine Namensauflösung über Hostnamen voraussetzt. **Überprüfung:** Derselbe Zugriff über den Hostnamen zeigt stattdessen Kerberos-Verkehr. **Fehleranalyse:** Kein Unterschied feststellbar? Prüfen, ob DNS tatsächlich korrekt konfiguriert ist. **Musterlösung:** Ein dokumentierter Vergleich zwischen Kerberos- und NTLM-Zugriff auf denselben Dienst.

7. Typische Fehler

Fehler	Ursache	Behebung
Kerberos- und NTLM-Verkehr verwechselt	Filtersyntax nicht genau geprüft	`kerberos` und `ntlmssp` als getrennte Filter nutzen
Mitschnitt am falschen Netzwerksegment	fehlende Übersicht über die Topologie	Mitschnittpunkt vorher mit dem Netzplan abgleichen
LDAP-Klartext-Risiko unterschätzt	Simple Bind als harmlos angenommen	Klartext-Bindungen aktiv auf Zugangsdaten prüfen

8. Zusammenfassung

Zusammenfassung

- Kerberos-Verkehr lässt sich anhand von AS-REQ/AS-REP/TGS-REQ/TGS-REP identifizieren.
- Unverschlüsselter LDAP kann bei einer Simple Bind Zugangsdaten im Klartext zeigen.
- Ein NTLM-Fallback tritt typischerweise bei fehlender Namensauflösung über Hostnamen auf.
- Mitschnitte bleiben ausschließlich auf das eigene, isolierte Labor beschränkt.

9. Kontrollfragen

1. Welcher Display-Filter zeigt Kerberos-Verkehr? (leicht)
2. Auf welchem Port läuft LDAPS? (leicht)
3. Warum kann eine Simple Bind ein Sicherheitsrisiko darstellen? (mittel)
4. Was ist eine typische Ursache für einen unerwarteten NTLM-Fallback? (mittel)
5. Ein Mitschnitt zeigt durchgehend Kerberos-Verkehr, aber keine einzige NTLM-Nachricht, über Wochen hinweg. Was lässt sich daraus über die DNS-Konfiguration der Umgebung schließen? (schwer)

Lösungshinweise: 1. `kerberos`. 2. 636. 3. Weil sie Zugangsdaten im Klartext übertragen kann, wenn keine zusätzliche Verschlüsselung erzwungen wird. 4. Fehlende oder fehlerhafte Namensauflösung über Hostnamen, etwa Zugriff über eine IP-Adresse. 5. Das deutet auf eine konsistent funktionierende DNS-Konfiguration hin, da Kerberos durchgehend genutzt werden kann, ohne auf das schwächere NTLM zurückzufallen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] Kerberos-Anmeldeverkehr erfolgreich mitgeschnitten hast,
- [ ] einen NTLM-Fallback gezielt provoziert und erkannt hast,
- [ ] den Unterschied zwischen LDAP und LDAPS erklären kannst,
- [ ] das Klartext-Risiko einer Simple Bind einschätzen kannst.

Kapitel 6: Windows-Sicherheitsanalyse mit Sysinternals

Lernziele

- Du nutzt Process Explorer zur detaillierten Prozessanalyse.
- Du findest Autostart-Einträge mit Autoruns.
- Du erkennst verdächtige Netzwerkverbindungen mit TCPView.

1. Einführung

Die Sysinternals-Sammlung, ursprünglich von Mark Russinovich entwickelt und heute Teil von Microsoft, liefert deutlich tiefere Einblicke in ein laufendes Windows-System als die Standardbordmittel.

2. Technische Grundlagen

Process Explorer zeigt laufende Prozesse mit deutlich mehr Detail als der Task-Manager, einschließlich geladener DLLs, Elternprozess-Hierarchie und digitaler Signaturprüfung. **Autoruns** listet sämtliche Mechanismen auf, über die Software automatisch beim Systemstart oder bei der Anmeldung geladen wird.

Beispiel 6.1: Process Explorer mit Signaturprüfung starten

```
procexp64.exe
Options -> Verify Image Signatures aktivieren
```

3. Fachbegriffe

TCPView zeigt aktive Netzwerkverbindungen mit zugehörigem Prozess in Echtzeit, vergleichbar mit `Get-NetTCPConnection`, aber mit laufender Aktualisierung. Eine nicht digital signierte, aber als Systemprozess getarnte Datei ist ein klassisches Warnsignal in Process Explorer.

4. Praxis und Konfiguration

Beispiel 6.2: Autostart-Einträge mit Autoruns prüfen

```
autorunsc64.exe -a * -c > autoruns_ergebnis.csv
```

Beispiel 6.3: Netzwerkverbindungen in Echtzeit beobachten

```
tcpview64.exe
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Lade Sysinternals-Werkzeuge ausschließlich von der offiziellen Microsoft-Quelle (live.sysinternals.com oder learn.microsoft.com). Gefälschte, nachgebaute Versionen bekannter Sicherheitswerkzeuge sind eine verbreitete Angriffsmethode.

6. Praktische Laborübungen

Übung A: Prozesse und Signaturen auf CLIENT01 prüfen

Aufgabenstellung: Starte Process Explorer und identifiziere alle nicht digital signierten, laufenden Prozesse. **Ausgangssituation:** CLIENT01, Standardkonfiguration. **Voraussetzungen:** Sysinternals-Suite heruntergeladen. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 6.1. **Erwartetes Ergebnis:** Eine Liste nicht signierter Prozesse, im Normalfall kurz oder leer. **Überprüfung:** Jeder gefundene, nicht signierte Prozess lässt sich einer bekannten, legitimen Anwendung zuordnen. **Fehleranalyse:** Viele nicht signierte Prozesse? Prüfen, ob es sich um Eigenentwicklungen oder tatsächlich verdächtige Software handelt. **Musterlösung:** Eine dokumentierte, vollständig erklärte Liste.

Übung B: Autostart-Einträge und Netzwerkverbindungen dokumentieren

Aufgabenstellung: Exportiere alle Autostart-Einträge und dokumentiere aktive Netzwerkverbindungen während normaler Nutzung. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 6.2, 6.3. **Erwartetes Ergebnis:** Eine vollständige CSV-Datei mit Autostart-Einträgen sowie eine Momentaufnahme aktiver Verbindungen. **Überprüfung:** Jeder Autostart-Eintrag lässt sich einer bekannten, installierten Anwendung zuordnen. **Fehleranalyse:** Unerklärlicher Eintrag? Dateipfad und Herausgeber in der Autoruns-Ausgabe genauer prüfen. **Musterlösung:** Eine vollständige, erklärte Baseline aus Autostart-Einträgen und Netzwerkverbindungen.

7. Typische Fehler

Fehler	Ursache	Behebung
Sysinternals von inoffizieller Quelle bezogen	fehlende Quellenprüfung	ausschließlich offizielle Microsoft-Quellen nutzen
Signaturprüfung in Process Explorer nicht aktiviert	Standardoption übersehen	Option bewusst in den Einstellungen aktivieren
Autoruns-Ausgabe nicht vollständig geprüft	sehr lange Liste überflogen statt gelesen	systematisch jeden unbekannten Eintrag einzeln prüfen

8. Zusammenfassung

Zusammenfassung

- Process Explorer bietet deutlich tiefere Prozesseinsicht als der Task-Manager, inklusive Signaturprüfung.
- Autoruns listet sämtliche Autostart-Mechanismen eines Systems vollständig auf.
- TCPView zeigt Netzwerkverbindungen mit Prozesszuordnung in Echtzeit.
- Sysinternals-Werkzeuge sollten ausschließlich von offiziellen Microsoft-Quellen bezogen werden.

9. Kontrollfragen

1. Was zeigt Process Explorer zusätzlich zum Task-Manager? (leicht)
2. Wozu dient Autoruns? (leicht)
3. Warum ist eine fehlende digitale Signatur ein Warnsignal, aber kein eindeutiger Beweis? (mittel)
4. Was unterscheidet TCPView von `Get-NetTCPConnection`? (mittel)
5. Ein Prozess trägt den Namen `svchost.exe`, ist aber nicht digital signiert und läuft aus einem ungewöhnlichen Verzeichnis. Was vermutest du, und wie gehst du vor? (schwer)

Lösungshinweise: 1. Geladene DLLs, Elternprozess-Hierarchie und digitale Signaturprüfung. 2. Es listet sämtliche Mechanismen auf, über die Software automatisch beim Start oder der Anmeldung geladen wird. 3. Weil auch legitime, aber selbst entwickelte oder seltene Software gelegentlich unsigniert sein kann. 4. TCPView aktualisiert laufend in Echtzeit mit grafischer Oberfläche, `Get-NetTCPConnection` liefert eine Momentaufnahme. 5. Ein echter `svchost.exe` läuft immer aus `C:\Windows\System32`; eine Variante aus einem anderen Verzeichnis ist ein klassisches Tarnmuster für Schadsoftware und verdient eine genauere Untersuchung, etwa über den Dateipfad und eine Signaturprüfung.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] Process Explorer mit Signaturprüfung eingesetzt hast,
- [ ] Autostart-Einträge mit Autoruns dokumentiert hast,
- [ ] Netzwerkverbindungen mit TCPView beobachtet hast,
- [ ] ein getarntes Systemprogramm anhand von Pfad und Signatur erkennen kannst.

Kapitel 7: Windows Privilege Escalation

Lernziele

- Du findest unsichere Dienstberechtigungen auf CLIENT02.
- Du nutzt ein automatisiertes Enumeration-Skript (WinPEAS) zur Fehlkonfigurationssuche.
- Du dokumentierst einen vollständigen Weg von eingeschränktem Zugang zu administrativen Rechten.

1. Einführung

Band 1, Kapitel 4 hat UAC und das Token-Konzept eingeführt. Dieses Kapitel setzt voraus, dass bereits ein eingeschränkter Zugang zu CLIENT02 besteht, und zeigt den Weg zu administrativen Rechten.

2. Technische Grundlagen

Unsichere Dienstberechtigungen entstehen, wenn ein eingeschränktes Konto einen mit höheren Rechten laufenden Dienst verändern oder dessen ausführbare Datei ersetzen kann. **Unquoted Service Paths** sind ein klassisches Muster: Ein Dienstpfad ohne Anführungszeichen mit Leerzeichen kann von Windows an mehreren, vom Angreifer kontrollierbaren Stellen nach einer ausführbaren Datei durchsucht werden.

Beispiel 7.1: Dienstberechtigungen und Pfade prüfen

```
PS> Get-CimInstance -ClassName Win32_Service | Select-Object Name, PathName, StartMode |
    Where-Object { $_.PathName -notmatch '^".*"$' -and $_.PathName -match ' ' }
```

3. Fachbegriffe

WinPEAS automatisiert die Suche nach genau solchen Fehlkonfigurationen: unsichere Dienstberechtigungen, unquoted Paths, AlwaysInstallElevated-Registry-Schlüssel, gespeicherte Zugangsdaten. **AlwaysInstallElevated** ist eine Registry-Einstellung, die MSI-Pakete grundsätzlich mit erhöhten Rechten installiert, unabhängig vom aufrufenden Konto.

4. Praxis und Konfiguration

Beispiel 7.2: WinPEAS auf CLIENT02 ausführen

```
PS> .\winPEASx64.exe > winpeas_ausgabe.txt
```

Beispiel 7.3: AlwaysInstallElevated prüfen

```
PS> Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Installer" `
    -Name AlwaysInstallElevated
PS> Get-ItemProperty -Path "HKCU:\SOFTWARE\Policies\Microsoft\Windows\Installer" `
    -Name AlwaysInstallElevated
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Jede Technik dieses Kapitels gehört ausschließlich gegen CLIENT02 in diesem Labor. Rechteausweitung setzt bereits einen bestehenden, unautorisierten Zugriff voraus, wenn sie gegen ein fremdes System eingesetzt wird.

6. Praktische Laborübungen

Übung A: Enumeration mit WinPEAS

Aufgabenstellung: Führe WinPEAS auf CLIENT02 aus und identifiziere den aussichtsreichsten Fund. **Ausgangssituation:** Eingeschränkter Zugang zu CLIENT02 (z. B. über das Testkonto aus dem Vorwort). **Voraussetzungen:** WinPEAS auf CLIENT02 übertragen. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 7.2. **Erwartetes Ergebnis:** Eine farblich markierte Ausgabe mit mindestens einem rot hervorgehobenen Fund. **Überprüfung:** Der Fund lässt sich einer bekannten Kategorie (Dienst, Registry, Pfad) zuordnen. **Fehleranalyse:** Keine Funde? Prüfen, ob CLIENT02 tatsächlich unverändert nach Vorwort-Installation ist. **Musterlösung:** Eine dokumentierte, priorisierte Liste der wichtigsten WinPEAS-Funde.

Übung B: Vollständige Rechteausweitung durchführen und dokumentieren

Aufgabenstellung: Nutze den Fund aus Übung A, um administrative Rechte zu erlangen, und dokumentiere den Weg. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** Beispiel 7.1 oder 7.3 entsprechend dem eigenen Fund anwenden. **Erwartetes Ergebnis:** Eine Sitzung mit administrativen Rechten auf CLIENT02. **Überprüfung:** `whoami /groups` zeigt die Administratoren-Gruppe als aktiv. **Fehleranalyse:** Kein Erfolg? Fund gegen eine öffentliche Referenz (z. B. HackTricks) zur Windows-Privilege-Escalation gegenprüfen. **Musterlösung:** Eine vollständige, nachvollziehbare Dokumentation von Ausgangszugang bis administrativer Shell.

7. Typische Fehler

Fehler	Ursache	Behebung
WinPEAS-Ausgabe nicht priorisiert	zu viele Funde gleichzeitig behandelt	zuerst rot, dann gelb markierte Funde prüfen
Unquoted-Path-Muster falsch interpretiert	Syntax nicht genau geprüft	Pfad exakt auf fehlende Anführungszeichen und Leerzeichen prüfen
Weg nicht dokumentiert	Zwischenschritte übersprungen	jeden Schritt von Anfang bis Ende festhalten

8. Zusammenfassung

Zusammenfassung

- Unsichere Dienstberechtigungen und unquoted Paths sind klassische Windows-Privilege-Escalation-Muster.
- WinPEAS automatisiert die Suche nach solchen Fehlkonfigurationen.

- AlwaysInstallElevated erlaubt MSI-Installationen grundsätzlich mit erhöhten Rechten.
- Jeder gefundene Weg sollte vollständig und nachvollziehbar dokumentiert werden.

9. Kontrollfragen

1. Was ist ein Unquoted Service Path? (leicht)
2. Welches Werkzeug automatisiert die Suche nach Windows-Privilege-Escalation-Fehlkonfigurationen? (leicht)
3. Was bewirkt AlwaysInstallElevated? (mittel)
4. Warum sind unsichere Dienstberechtigungen ein typisches Einfallstor? (mittel)
5. Du findest auf CLIENT02 sowohl einen Unquoted Service Path als auch eine unsichere Dienstberechtigung. Welchen Weg dokumentierst du zuerst, und warum könnte die Reihenfolge für einen Bericht relevant sein? (schwer)

Lösungshinweise: 1. Ein Dienstpfad ohne Anführungszeichen mit Leerzeichen, der von Windows an mehreren Stellen nach einer ausführbaren Datei durchsucht wird. 2. WinPEAS. 3. Es lässt MSI-Pakete grundsätzlich mit erhöhten Rechten installieren, unabhängig vom aufrufenden Konto. 4. Weil ein Dienst, der von einem eingeschränkten Konto verändert werden kann, aber mit höheren Rechten läuft, einen direkten Weg zu diesen Rechten eröffnet. 5. Beide sollten dokumentiert werden; die Reihenfolge im Bericht richtet sich sinnvollerweise nach Zuverlässigkeit und Einfachheit, damit ein Auftraggeber den am leichtesten zu behebenden oder kritischsten Weg zuerst sieht.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] WinPEAS erfolgreich gegen CLIENT02 ausgeführt hast,
- [ ] einen gefundenen Weg zur Rechteausweitung erfolgreich genutzt hast,
- [ ] den vollständigen Weg nachvollziehbar dokumentiert hast,
- [ ] Unquoted Service Paths und AlwaysInstallElevated erklären kannst.

Kapitel 8: Active Directory Security Assessment

Lernziele

- Du nutzt BloodHound zur Visualisierung von Angriffspfaden.
- Du identifizierst einen mehrstufigen Pfad zu Domain Admin.
- Du bewertest einen gefundenen Pfad als Grundlage für eine Empfehlung.

1. Einführung

Kapitel 3 hat gezeigt, wie viel ein einzelnes Konto preisgibt. Dieses Kapitel verbindet diese Einzelinformationen zu einem vollständigen Bild möglicher Angriffspfade durch die gesamte Domäne.

2. Technische Grundlagen

BloodHound sammelt AD-Beziehungsdaten (Gruppenmitgliedschaften, Sitzungen, Berechtigungen) über einen Collector (z. B. SharpHound) und stellt sie als durchsuchbaren Graphen dar. Oft führt kein einzelner, offensichtlicher Fehler zu Domain Admin, sondern eine Kette mehrerer, für sich genommen harmloser Berechtigungen.

Beispiel 8.1: Daten mit SharpHound sammeln

```
PS> .\SharpHound.exe -c All
```

3. Fachbegriffe

Ein **Angriffspfad** (Attack Path) ist eine Kette von Berechtigungsbeziehungen, die von einem Ausgangskonto zu einem hochprivilegierten Ziel führt. **Shortest Path to Domain Admins** ist eine Standardabfrage in BloodHound, die automatisch den kürzesten gefundenen Pfad anzeigt.

4. Praxis und Konfiguration

Beispiel 8.2: Gesammelte Daten in BloodHound importieren und analysieren

```
BloodHound-GUI -> Daten importieren -> Vordefinierte Abfrage:
"Shortest Paths to Domain Admins from Owned Principals"
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

BloodHound-Datensammlung erzeugt erheblichen LDAP- und teils Netzwerkverkehr. Setze es ausschließlich gegen die eigene Labor-Domäne ein, nie gegen eine produktive oder fremde Umgebung ohne ausdrücklichen Auftrag.

Ein gefundener Angriffspfad ist oft keine einzelne „Schwachstelle“ im klassischen Sinn, sondern das Ergebnis mehrerer, für sich genommen sinnvoller administrativer Entscheidungen, die in Kombination ein Risiko ergeben – ein Grund, warum Delegation (Band 2, Kapitel 5) und regelmäßige Revision privilegierter Gruppen so wichtig sind.

6. Praktische Laborübungen

Übung A: Daten sammeln und in BloodHound importieren

Aufgabenstellung: Sammle Daten mit SharpHound und importiere sie in BloodHound. **Ausgangssituation:** Die Labor-Domäne aus Band 2, ein authentifiziertes Testkonto. **Voraussetzungen:** SharpHound und BloodHound auf KALI01 oder einem Windows-System verfügbar. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 8.1, 8.2. **Erwartetes Ergebnis:** Ein vollständig geladener Graph der Domänenstruktur. **Überprüfung:** Die Anzahl der importierten Benutzer und Gruppen stimmt mit der tatsächlichen Domäne überein. **Fehleranalyse:** Import schlägt fehl? Dateiformat und Vollständigkeit der SharpHound-Ausgabe prüfen. **Musterlösung:** Ein vollständig geladener, durchsuchbarer Graph.

Übung B: Einen Angriffspfad zu Domain Admin finden und bewerten

Aufgabenstellung: Führe die Abfrage aus Beispiel 8.2 aus und bewerte den gefundenen kürzesten Pfad. **Ausgangssituation:** Ergebnis aus Übung A, das Testkonto als „Owned“ markiert. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 8.2, danach jeden einzelnen Schritt des gefundenen Pfades nachvollziehen. **Erwartetes Ergebnis:** Ein visualisierter, mehrstufiger Pfad vom Testkonto zu Domain Admins, sofern einer existiert. **Überprüfung:** Jeder Schritt des Pfades lässt sich inhaltlich erklären (z. B. Gruppenmitgliedschaft, Delegation). **Fehleranalyse:** Kein Pfad gefunden? Das ist ein gutes, dokumentierbares Ergebnis für eine gut abgesicherte Domäne. **Musterlösung:** Eine vollständige Dokumentation des gefundenen Pfades mit einer konkreten Empfehlung, welches Glied der Kette zu durchbrechen wäre.

7. Typische Fehler

Fehler	Ursache	Behebung
BloodHound gegen fremde Umgebung eingesetzt	Scope nicht beachtet	ausschließlich gegen die eigene Labor-Domäne
Pfad gefunden, aber nicht erklärt	Visualisierung ohne inhaltliches Verständnis übernommen	jeden Schritt einzeln nachvollziehen und begründen
Einzelne Berechtigung isoliert statt im Zusammenhang bewertet	Kettenwirkung übersehen	Angriffspfade als Gesamtkette betrachten

8. Zusammenfassung

Zusammenfassung

- BloodHound visualisiert AD-Berechtigungsbeziehungen als durchsuchbaren Graphen.
- Angriffspfade entstehen oft aus einer Kette mehrerer, für sich harmloser Berechtigungen.
- Die Standardabfrage „Shortest Path to Domain Admins“ zeigt den kürzesten gefundenen Weg.
- Jeder gefundene Pfad sollte Schritt für Schritt erklärt, nicht nur visualisiert werden.

9. Kontrollfragen

1. Was sammelt SharpHound? (leicht)
2. Was visualisiert BloodHound? (leicht)
3. Warum ist ein Angriffspfad oft keine einzelne Schwachstelle? (mittel)
4. Was zeigt die Abfrage „Shortest Path to Domain Admins“? (mittel)
5. Ein gefundener Angriffspfad führt über drei verschachtelte Gruppen zu Domain Admin, jede für sich genommen sinnvoll konfiguriert. Wie würdest du die Empfehlung formulieren, ohne eine legitime Struktur unnötig zu zerschlagen? (schwer)

Lösungshinweise: 1. AD-Beziehungsdaten wie Gruppenmitgliedschaften, Sitzungen und Berechtigungen. 2. Berechtigungsbeziehungen in Active Directory als durchsuchbaren Graphen. 3. Weil sie oft aus mehreren, für sich genommen harmlosen Berechtigungen in Kombination entsteht. 4. Den kürzesten gefundenen Pfad von einem markierten Ausgangskonto zur Gruppe Domain Admins. 5. Statt die gesamte Verschachtelung aufzulösen, gezielt das schwächste, am leichtesten zu entfernende Glied der Kette identifizieren (etwa eine nicht mehr benötigte Gruppenmitgliedschaft) und nur dieses eine Glied als konkrete, minimal-invasive Empfehlung vorschlagen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] Daten mit SharpHound gesammelt und in BloodHound importiert hast,
- [ ] einen Angriffspfad zu Domain Admin gefunden und erklärt hast,
- [ ] eine konkrete, minimal-invasive Empfehlung formuliert hast,
- [ ] erklären kannst, warum Angriffspfade oft aus mehreren, einzeln harmlosen Schritten bestehen.

Kapitel 9: Kerberos-Angriffe und Schutzmaßnahmen

Lernziele

- Du erklärst das Prinzip von Kerberoasting.
- Du führst einen Kerberoasting-Test gegen ein Dienstkonto der Labor-Domäne durch.
- Du härtest ein Dienstkonto gezielt gegen diese Angriffsklasse.

1. Einführung

Band 2, Kapitel 7 hat Kerberos als sicheres Protokoll vorgestellt. Dieses Kapitel zeigt, warum selbst ein grundsätzlich sicheres Protokoll durch schwache Dienstkonto-Passwörter angreifbar wird.

2. Technische Grundlagen

Kerberoasting nutzt aus, dass jedes authentifizierte Domänenkonto ein Dienst-Ticket (Kapitel 5, TGS) für einen beliebigen Service Principal Name (SPN) anfordern darf. Dieses Ticket ist teilweise mit dem Passwort-Hash des Dienstkontos verschlüsselt – ein Angreifer kann es offline gegen eine Wortliste testen, ohne dass dafür eine Fehlanmeldung im System sichtbar würde.

Beispiel 9.1: Dienstkonten mit SPN auflisten

```
PS> Get-ADUser -Filter {ServicePrincipalName -ne "$null"} -Properties ServicePrincipalName
```

3. Fachbegriffe

Ein **SPN** (Service Principal Name) identifiziert einen Dienst eindeutig innerhalb der Domäne und verknüpft ihn mit dem ausführenden Konto. **AS-REP Roasting** ist eine verwandte, aber auf eine andere Fehlkonfiguration (fehlende Vorauthentifizierung) zielende Technik.

4. Praxis und Konfiguration

Beispiel 9.2: Ein Kerberos-Dienst-Ticket anfordern und offline speichern (nur eigenes Testkonto!)

```
PS> Add-Type -AssemblyName System.IdentityModel
PS> New-Object System.IdentityModel.Tokens.KerberosRequestorSecurityToken `
    -ArgumentList "MSSQLSvc/srv01.nova.int:1433"
```

Gut zu wissen

Beispiel 9.2 fordert das Ticket an und legt es im Kerberos-Ticketspeicher der laufenden Sitzung ab, noch nicht als Datei. Um daraus tatsächlich die in Beispiel 9.3 benötigte `ticket.hash` zu erhalten, müssen die rohen Ticket-Bytes aus diesem Speicher extrahiert und in das von Hashcat erwartete Format umgewandelt werden. Genau dafür existieren spezialisierte Werkzeuge wie Rubeus oder Invoke-Kerberoast, die Anforderung, Extraktion und Formatumwandlung in einem Schritt bündeln. Dieses Buch zeigt bewusst nur die beiden konzeptionellen Teilschritte, Anfrage und Offline-Test, nicht eine fertige, direkt einsatzbereite Werkzeugkette.

Beispiel 9.3: Das gewonnene Ticket offline gegen eine Wortliste prüfen (Konzept, vergleiche John-the-Ripper-/Hashcat-Werkzeuge der Linux-Schwesterreihe)

```
$ hashcat -a 0 -m 13100 ticket.hash rockyou.txt
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Führe Kerberoasting ausschließlich gegen ein eigens dafür angelegtes Testdienstkonto in der Labor-Domäne durch, niemals gegen echte Produktivkonten. Dieselbe Technik gegen eine fremde Domäne wäre, unabhängig vom technischen Weg, ein eigenständiger, schwerer Rechtsverstoß.

6. Praktische Laborübungen

Übung A: Dienstkonten mit SPN identifizieren

Aufgabenstellung: Liste alle Dienstkonten mit SPN in der Labor-Domäne auf. **Ausgangssituation:** Domäne aus Band 2, ggf. mit einem eigens angelegten Testdienstkonto. **Voraussetzungen:** authentifizierter Domänenzugang. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 9.1. **Erwartetes Ergebnis:** Eine vollständige Liste der Dienstkonten mit ihrem jeweiligen SPN. **Überprüfung:** Jedes gelistete Konto lässt sich einem bekannten Dienst zuordnen. **Fehleranalyse:** Keine Treffer? Ein Testdienstkonto mit SPN nach Band 2, Kapitel 12 anlegen. **Musterlösung:** Eine vollständige, dokumentierte Liste.

Übung B: Kerberoasting gegen ein Testkonto durchführen und das Konto härten

Aufgabenstellung: Führe das Grundprinzip aus Beispiel 9.2/9.3 gegen ein eigens angelegtes Testdienstkonto mit absichtlich schwachem Passwort durch, härte danach das Konto mit einem starken, zufälligen Passwort, und wiederhole den Test. **Ausgangssituation:** Ergebnis aus Übung A, ein Testdienstkonto mit schwachem Passwort. **Voraussetzungen:** wie Übung A; ein Werkzeug, das Anfrage, Extraktion und Formatumwandlung bündelt (z. B. Rubeus oder Invoke-Kerberoast), da die reinen .NET-Beispiele diese Schritte einzeln nicht abdecken (siehe Hinweis nach Beispiel 9.2). **Schritt-für-Schritt-Anleitung:** Ticket mit dem gewählten Werkzeug anfordern und extrahieren, danach mit Beispiel 9.3 offline testen, danach Passwort auf mindestens 25 zufällige Zeichen ändern (oder auf ein gMSA umstellen, Band 2, Kapitel 12), Test wiederholen. **Erwartetes Ergebnis:** Das schwache Passwort wird gefunden, das starke oder gMSA-basierte Konto bleibt innerhalb der Testzeit unentschlüsselt. **Überprüfung:** Beide Ergebnisse lassen sich direkt gegenüberstellen. **Fehleranalyse:** Auch das starke Passwort gefunden? Prüfen, ob es tatsächlich die geforderte Länge und Zufälligkeit hat. **Musterlösung:** Ein dokumentierter Vorher-Nachher-Vergleich, der die Wirksamkeit starker Dienstkonto-Passwörter oder gMSAs gegen Kerberoasting belegt.

7. Typische Fehler

Fehler	Ursache	Behebung
Kerberoasting gegen echtes Produktivkonto getestet	fehlende Eingrenzung	ausschließlich eigens angelegte Testkonten verwenden
Dienstkonto-Passwort trotz Kenntnis des Risikos nie geändert	fehlende Priorisierung	starke Passwörter oder gMSAs als Standard etablieren
SPN-Konten nie systematisch erfasst	fehlende Routine	regelmäßig mit Beispiel 9.1 prüfen

8. Zusammenfassung

> **Zusammenfassung**
>
> - Kerberoasting nutzt aus, dass jedes Konto ein Dienst-Ticket für einen beliebigen SPN anfordern darf.
> - Das Ticket lässt sich offline, ohne sichtbare Fehlanmeldung, gegen eine Wortliste prüfen.
> - Starke, zufällige Dienstkonto-Passwörter oder gMSAs machen diesen Angriff praktisch wirkungslos.
> - Die Technik gehört ausschließlich gegen eigens angelegte Testkonten im eigenen Labor.

9. Kontrollfragen

1. Was ist ein SPN? (leicht)
2. Welcher Befehl listet Dienstkonten mit SPN auf? (leicht)
3. Warum ist Kerberoasting ohne sichtbare Fehlanmeldung durchführbar? (mittel)
4. Was unterscheidet Kerberoasting von AS-REP Roasting? (mittel)
5. Ein Unternehmen nutzt für einen Dienst ein klassisches Dienstkonto mit einem seit Jahren unveränderten, aber „ausreichend langen“ Passwort. Reicht das gegen Kerberoasting aus? Begründe. (schwer)

Lösungshinweise: 1. Service Principal Name, eine eindeutige Dienstkennung innerhalb der Domäne. 2. `Get-ADUser -Filter {ServicePrincipalName -ne "$null"}`. 3. Weil die eigentliche Knackarbeit offline, ohne weitere Anfragen an den Domain Controller erfolgt. 4. Kerberoasting zielt auf TGS-Tickets über reguläre Dienstanfragen, AS-REP Roasting auf Konten ohne Kerberos-Vorauthentifizierung. 5. Allein die Länge reicht nicht aus; entscheidend ist zusätzlich echte Zufälligkeit (kein Muster, kein Wörterbuchwort) und idealerweise die Umstellung auf ein gMSA, das Passwortverwaltung vollständig automatisiert.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] Dienstkonten mit SPN identifiziert hast,
- [ ] einen Kerberoasting-Test gegen ein Testkonto durchgeführt hast,
- [ ] das Konto erfolgreich gegen diesen Angriff gehärtet hast,
- [ ] den Unterschied zwischen Kerberoasting und AS-REP Roasting erklären kannst.

Kapitel 10: NTLM-Sicherheitsrisiken

Lernziele

- Du erklärst das Prinzip eines NTLM-Relay-Angriffs.
- Du erkennst, welche Konfigurationen NTLM-Relay in der Labor-Domäne begünstigen.
- Du setzt SMB-Signierung als konkrete Gegenmaßnahme durch.

1. Einführung

Band 2, Kapitel 7 hat NTLM als schwächer gegenüber Kerberos eingeordnet. Dieses Kapitel zeigt konkret, warum: NTLM-Relay gehört zu den am häufigsten in realen Vorfällen genutzten Techniken gegen Windows-Domänen.

2. Technische Grundlagen

Ein **NTLM-Relay-Angriff** leitet eine abgefangene NTLM-Authentifizierungsanfrage an ein anderes System weiter, statt sie selbst zu beantworten – gelingt dies, authentifiziert sich das Opfer unwissentlich gegenüber dem vom Angreifer gewählten Ziel. Ohne SMB-Signierung (Band 2, Kapitel 8) lässt sich eine weitergeleitete Authentifizierung vom Zielsystem nicht von einer echten, direkten Verbindung unterscheiden.

Beispiel 10.1: Grundprinzip eines NTLM-Relay-Angriffs (Konzept)

```
Opfer -> authentifiziert sich bei einem vom Angreifer kontrollierten System
Angreifer -> leitet dieselbe Authentifizierung an SRV01 weiter
SRV01 -> akzeptiert die weitergeleitete Authentifizierung als "vom Opfer kommend"
```

3. Fachbegriffe

LLMNR und **NBT-NS** sind veraltete, lokale Namensauflösungsprotokolle, die bei fehlgeschlagener DNS-Auflösung automatisch im lokalen Netzwerksegment nach einem Namen fragen – ein Angreifer kann auf diese Anfrage antworten und so Authentifizierungsversuche auf sich umleiten. **Responder** ist ein verbreitetes, öffentlich dokumentiertes Werkzeug, das genau diese Anfragen beantwortet.

4. Praxis und Konfiguration

Beispiel 10.2: LLMNR und NBT-NS deaktivieren (per GPO)

```
Computerkonfiguration -> Richtlinien -> Administrative Vorlagen -> Netzwerk ->
DNS-Client -> "Multicast-Namensauflösung (LLMNR) deaktivieren" -> Aktiviert
```

Beispiel 10.3: SMB-Signierung erzwingen (Wiederholung aus Band 2, Kapitel 8)

```
Computerkonfiguration -> Sicherheitseinstellungen -> Lokale Richtlinien ->
Sicherheitsoptionen ->
"Microsoft-Netzwerk (Server): Kommunikation digital signieren (immer)" -> Aktiviert
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Setze Werkzeuge zur NTLM-Relay-Demonstration ausschließlich gegen die eigene Labor-Domäne ein. Das Abfangen und Umleiten fremder Authentifizierungsanfragen ist unabhängig vom Ergebnis ein schwerer, eigenständiger Rechtsverstoß.

6. Praktische Laborübungen

Übung A: LLMNR/NBT-NS-Nutzung in der Domäne prüfen und deaktivieren

Aufgabenstellung: Prüfe, ob LLMNR/NBT-NS in der Labor-Domäne aktiv sind, und deaktiviere sie per GPO. **Ausgangssituation:** Domäne aus Band 2. **Voraussetzungen:** administrative Domänenrechte. **Schritt-für-Schritt-Anleitung:** Einen absichtlich falschen Hostnamen von CLIENT02 aus auflösen lassen und mit Wireshark (Kapitel 5) auf LLMNR-Anfragen prüfen, danach Beispiel 10.2 anwenden. **Erwartetes Ergebnis:** Vor der Änderung erscheinen LLMNR-Anfragen im Mitschnitt, danach nicht mehr. **Überprüfung:** Ein erneuter Test nach `gpupdate /force` zeigt keine LLMNR-Anfragen mehr. **Fehleranalyse:** Weiterhin Anfragen sichtbar? GPO-Verknüpfung und Anwendung auf CLIENT02 prüfen. **Musterlösung:** Ein dokumentierter Vorher-Nachher-Vergleich der LLMNR-Nutzung.

Übung B: SMB-Signierung als Gegenmaßnahme bestätigen

Aufgabenstellung: Erzwinge SMB-Signierung nach Beispiel 10.3 und erkläre, warum das NTLM-Relay gegen SMB-Ziele erschwert. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 10.3, danach schriftlich erklären, an welcher Stelle des Ablaufs aus Beispiel 10.1 die Signierung eingreift. **Erwartetes Ergebnis:** Eine korrekte, schriftliche Erklärung, dass eine signierte Verbindung die Herkunft der Authentifizierung prüfbar macht. **Überprüfung:** Die Erklärung bezieht sich konkret auf den Ablauf aus Beispiel 10.1. **Fehleranalyse:** Unklare Erklärung? Band 2, Kapitel 8 zur SMB-Signierung erneut konsultieren. **Musterlösung:** Eine korrekte, technisch begründete Erklärung der Schutzwirkung.

7. Typische Fehler

Fehler	Ursache	Behebung
LLMNR/NBT-NS als harmlose Altlast abgetan	Risiko unterschätzt	aktiv deaktivieren, wo nicht zwingend benötigt
SMB-Signierung nur auf einem System erzwungen	GPO nicht domänenweit angewendet	Verknüpfung auf alle relevanten Systeme ausweiten
NTLM-Relay-Werkzeuge gegen reale Systeme getestet	Scope nicht beachtet	ausschließlich im eigenen Labor einsetzen

8. Zusammenfassung

Zusammenfassung

- NTLM-Relay leitet eine abgefangene Authentifizierung an ein anderes Ziel weiter.
- LLMNR und NBT-NS erleichtern Angreifern, Authentifizierungsversuche auf sich umzuleiten.
- SMB-Signierung macht die Herkunft einer Verbindung prüfbar und erschwert Relay-Angriffe erheblich.

> - Jede Demonstrationstechnik bleibt ausschließlich im eigenen, isolierten Labor.

9. Kontrollfragen

1. Was ist ein NTLM-Relay-Angriff? (leicht)
2. Wofür stehen LLMNR und NBT-NS? (leicht)
3. Warum erschwert SMB-Signierung NTLM-Relay? (mittel)
4. Welches öffentlich bekannte Werkzeug wird häufig zur Demonstration von LLMNR-Missbrauch genannt? (mittel)
5. Ein Unternehmen hat SMB-Signierung erzwungen, aber LLMNR weiterhin aktiv. Ist das Risiko damit vollständig behoben? (schwer)

Lösungshinweise: 1. Ein Angriff, der eine abgefangene NTLM-Authentifizierung an ein anderes System weiterleitet. 2. Link-Local Multicast Name Resolution und NetBIOS Name Service. 3. Weil eine signierte Verbindung die Herkunft prüfbar macht, sodass eine weitergeleitete Authentifizierung erkennbar wird. 4. Responder. 5. Nicht vollständig: SMB-Signierung schützt gezielt SMB-Ziele, LLMNR bleibt aber weiterhin ein Weg, Authentifizierungsversuche überhaupt erst abzufangen, auch wenn deren Weiterleitung gegen signierte SMB-Ziele dann erschwert ist; beide Maßnahmen ergänzen sich.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] LLMNR/NBT-NS-Nutzung geprüft und deaktiviert hast,
- [ ] SMB-Signierung als konkrete Gegenmaßnahme erzwungen hast,
- [ ] das Grundprinzip eines NTLM-Relay-Angriffs erklären kannst,
- [ ] erklären kannst, warum mehrere Gegenmaßnahmen sich ergänzen statt sich zu ersetzen.

Kapitel 11: Passwortsicherheit und Hashverfahren

Lernziele

- Du erklärst, wie Windows Passwort-Hashes speichert (NTLM-Hash, SAM).
- Du extrahierst Hashes autorisiert aus einer Testumgebung.
- Du bewertest den Geschwindigkeitsunterschied zwischen NTLM- und Kerberos-Hash-Verfahren.

1. Einführung

Kapitel 9 hat Kerberos-Tickets geknackt, nicht direkt lokale Passwort-Hashes. Dieses Kapitel vertieft die Grundlage: wie Windows Passwörter überhaupt speichert, und warum der NTLM-Hash selbst als schwach gilt.

2. Technische Grundlagen

Lokale Windows-Passwörter werden als **NTLM-Hash** in der **SAM**-Datenbank (Security Account Manager) gespeichert. Der NTLM-Hash ist ungesalzen und kryptografisch schwach (im Kern ein einfacher MD4-Hash), weshalb identische Passwörter auf verschiedenen Konten stets denselben Hash ergeben – ein deutlicher Unterschied zu den gesalzenen Verfahren der Linux-Schwesterreihe.

Beispiel 11.1: SAM-Datenbank-Struktur (Konzept, nicht direkt zugreifbar im laufenden Betrieb)

```
Die SAM-Datei ist gesperrt, solange Windows läuft; Zugriff erfordert
entweder administrative Rechte zur Registry-Hive-Extraktion oder
Offline-Zugriff über ein Boot-Medium.
```

3. Fachbegriffe

Pass-the-Hash nutzt aus, dass NTLM zur Authentifizierung den Hash selbst verwendet, nicht das Klartext-Passwort – ein gestohlener Hash reicht aus, ohne ihn jemals zu entschlüsseln. **NTDS.dit** ist das AD-Pendant zur lokalen SAM-Datenbank und speichert Hashes aller Domänenkonten auf dem Domain Controller.

4. Praxis und Konfiguration

Beispiel 11.2: Lokale Hashes auf dem eigenen Testsystem mit administrativen Rechten extrahieren (nur eigenes Testkonto)

```
PS> reg save HKLM\SAM sam_backup.hiv
PS> reg save HKLM\SYSTEM system_backup.hiv
```

Beispiel 11.3: Geschwindigkeit von NTLM gegen ein Kerberos-Hash-Format vergleichen (vergleiche Linux-Schwesterreihe, Hashcat-Handbuch)

```
$ hashcat -b -m 1000    # NTLM
$ hashcat -b -m 13100   # Kerberos 5 TGS-REP
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Das Extrahieren von Passwort-Hashes ist ausschließlich auf einem eigenen Testsystem mit eigenen, dafür vorgesehenen Testkonten zulässig. Der Zugriff auf `NTDS.dit` oder die SAM-Datenbank eines fremden Systems ohne Autorisierung ist unabhängig von der technischen Methode strafbar.

6. Praktische Laborübungen

Übung A: NTLM-Hash-Verhalten bei identischen Passwörtern beobachten

Aufgabenstellung: Lege zwei Testkonten mit identischem Passwort an und vergleiche ihre NTLM-Hashes. **Ausgangssituation:** CLIENT02, administrative Testrechte. **Voraussetzungen:** zwei Testkonten, ein Werkzeug zur Hash-Extraktion (z. B. Mimikatz, ausschließlich im eigenen Labor). **Schritt-für-Schritt-Anleitung:** Zwei Konten mit identischem Passwort anlegen, Hashes extrahieren, vergleichen. **Erwartetes Ergebnis:** Beide Konten zeigen denselben NTLM-Hash. **Überprüfung:** Ein Unterschied zum gesalzenen Verhalten unter Linux lässt sich klar benennen. **Fehleranalyse:** Unterschiedliche Hashes trotz identischem Passwort? Tippfehler beim Anlegen der Testpasswörter ausschließen. **Musterlösung:** Eine dokumentierte Bestätigung des fehlenden Saltings bei NTLM.

Übung B: Geschwindigkeitsunterschied zwischen Hash-Verfahren messen

Aufgabenstellung: Führe Beispiel 11.3 aus und ordne das Ergebnis praktisch ein. **Ausgangssituation:** Hashcat auf KALI01 installiert. **Voraussetzungen:** keine besonderen. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 11.3. **Erwartetes Ergebnis:** NTLM zeigt eine deutlich höhere Hashrate als das Kerberos-TGS-Format. **Überprüfung:** Der Unterschied lässt sich mit dem jeweiligen kryptografischen Aufwand erklären. **Fehleranalyse:** Kein GPU-Gerät erkannt? Mit `-D 1` den CPU-Modus erzwingen. **Musterlösung:** Eine dokumentierte Gegenüberstellung beider Benchmark-Werte mit Einordnung.

7. Typische Fehler

Fehler	Ursache	Behebung
NTLM-Hash für „verschlüsselt und sicher“ gehalten	Begriffe nicht klar unterschieden	NTLM-Hash ist ungesalzen und kryptografisch schwach
Pass-the-Hash mit Passwort-Diebstahl verwechselt	Unterschied nicht verstanden	Pass-the-Hash nutzt den Hash selbst, ohne ihn zu entschlüsseln
Hash-Extraktion gegen fremdes System versucht	Scope nicht beachtet	ausschließlich eigene Testsysteme verwenden

8. Zusammenfassung

Zusammenfassung

- Windows speichert lokale Passwörter als ungesalzenen NTLM-Hash in der SAM-Datenbank.
- Identische Passwörter ergeben bei NTLM stets denselben Hash, anders als bei gesalzenen Linux-Verfahren.
- Pass-the-Hash nutzt den gestohlenen Hash direkt zur Authentifizierung, ohne ihn zu entschlüsseln.

- NTDS.dit speichert Hashes aller Domänenkonten auf dem Domain Controller.

9. Kontrollfragen

1. Wo speichert Windows lokale Passwort-Hashes? (leicht)
2. Was ist ein Pass-the-Hash-Angriff? (leicht)
3. Warum ergeben identische Passwörter bei NTLM immer denselben Hash? (mittel)
4. Was ist NTDS.dit? (mittel)
5. Ein Administrator argumentiert, NTLM-Hashes seien „ausreichend sicher, weil sie nicht im Klartext gespeichert werden“. Was entgegnest du mit dem Wissen aus diesem Kapitel? (schwer)

Lösungshinweise: 1. In der SAM-Datenbank (Security Account Manager). 2. Ein Angriff, der einen gestohlenen Passwort-Hash direkt zur Authentifizierung nutzt, ohne ihn zu entschlüsseln. 3. Weil NTLM kein Salting verwendet. 4. Das AD-Pendant zur lokalen SAM-Datenbank, speichert Hashes aller Domänenkonten auf dem Domain Controller. 5. Hashing allein schützt nicht ausreichend, wenn das Verfahren ungesalzen und kryptografisch schwach ist; zusätzlich ermöglicht Pass-the-Hash die Nutzung des Hashes selbst zur Authentifizierung, ganz ohne ihn jemals zu entschlüsseln.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] das fehlende Salting von NTLM-Hashes praktisch beobachtet hast,
- [ ] den Geschwindigkeitsunterschied zwischen Hash-Verfahren gemessen hast,
- [ ] Pass-the-Hash als Konzept erklären kannst,
- [ ] den Unterschied zwischen SAM und NTDS.dit benennen kannst.

Kapitel 12: Windows-Schwachstellenanalyse

Lernziele

- Du führst einen automatisierten Schwachstellenscan gegen CLIENT02 durch.
- Du ordnest einen gefundenen Treffer gegen eine echte, öffentliche CVE ein.
- Du unterscheidest einen authentifizierten von einem nicht authentifizierten Scan.

1. Einführung

Die vorherigen Kapitel haben gezielt einzelne Fehlerklassen gesucht. Dieses Kapitel ergänzt einen automatisierten, breiten Scanansatz, wie er in professionellen Audits üblich ist.

2. Technische Grundlagen

Ein **nicht authentifizierter Scan** prüft ein System von außen, ohne Zugangsdaten, und findet vor allem netzwerkseitig sichtbare Schwachstellen. Ein **authentifizierter Scan** meldet sich mit Zugangsdaten am System an und kann dadurch deutlich mehr finden, etwa fehlende Patches auf Anwendungsebene oder lokale Fehlkonfigurationen.

Beispiel 12.1: Nmap-Vulnerability-Skripte gegen CLIENT02

```
$ sudo nmap --script=vuln -p 445 192.168.56.60
```

3. Fachbegriffe

OpenVAS (bzw. Greenbone) ist ein verbreiteter, quelloffener Schwachstellenscanner, der sowohl authentifizierte als auch nicht authentifizierte Scans unterstützt. Ein **False Positive** ist ein gemeldeter Fund, der sich bei näherer Prüfung als nicht tatsächlich zutreffend herausstellt – bei automatisierten Scans nicht selten.

4. Praxis und Konfiguration

Beispiel 12.2: Einen authentifizierten Scan mit OpenVAS vorbereiten (Konzept)

```
GSM-Weboberfläche -> Konfiguration -> Zugangsdaten ->
Neue Zugangsdaten für CLIENT02 hinterlegen -> Scan-Ziel mit Zugangsdaten verknüpfen
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Ein automatisierter Vulnerability-Scan kann auf instabilen oder schlecht gewarteten Diensten unerwartete Abstürze auslösen. Führe Scans im Labor zunächst gegen CLIENT02 durch, nicht gegen DC01, wo ein unerwarteter Ausfall die gesamte Domäne beeinträchtigen würde.

6. Praktische Laborübungen

Übung A: Nicht authentifizierten Scan durchführen

Aufgabenstellung: Führe Beispiel 12.1 gegen CLIENT02 durch und dokumentiere die Funde. **Ausgangssituation:** CLIENT02 nach Vorwort-Installation. **Voraussetzungen:** Nmap auf KALI01. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 12.1. **Erwartetes Ergebnis:** Eine Liste potenzieller Schwachstellen, teils mit CVE-Referenz. **Überprüfung:** Jeder Treffer lässt sich gegen eine öffentliche Quelle (z. B. NVD) nachschlagen. **Fehleranalyse:** Keine Treffer? Normal bei einem aktuell gepatchten System; ergänzend `--script=smb-vuln*` gezielt testen. **Musterlösung:** Eine dokumentierte Liste mit Bewertung jedes Treffers.

Übung B: Authentifizierten Scan vorbereiten und Unterschied bewerten

Aufgabenstellung: Richte einen authentifizierten Scan mit OpenVAS ein (sofern verfügbar) und vergleiche den Funktionsumfang mit Übung A. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** OpenVAS/Greenbone installiert, Testzugangsdaten für CLIENT02. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 12.2, danach Scan starten und Ergebnisse vergleichen. **Erwartetes Ergebnis:** Der authentifizierte Scan findet deutlich mehr, darunter lokale Konfigurationsfehler. **Überprüfung:** Mindestens ein Fund ist ausschließlich im authentifizierten Scan sichtbar. **Fehleranalyse:** Kein Unterschied feststellbar? Zugangsdaten-Konfiguration in OpenVAS erneut prüfen. **Musterlösung:** Eine dokumentierte Gegenüberstellung beider Scan-Arten mit konkreten Beispielen.

7. Typische Fehler

Fehler	Ursache	Behebung
Vulnerability-Scan direkt gegen DC01 gestartet	Risiko für die gesamte Domäne unterschätzt	zuerst gegen weniger kritische Systeme testen
Jeder automatisierte Fund ungeprüft übernommen	False Positives nicht berücksichtigt	jeden Fund manuell gegenprüfen
Authentifizierter und nicht authentifizierter Scan verwechselt	Begriffe nicht klar unterschieden	Unterschied im Funktionsumfang aktiv nachvollziehen

8. Zusammenfassung

Zusammenfassung

- Nicht authentifizierte Scans prüfen von außen, authentifizierte mit Zugangsdaten deutlich tiefer.
- Nmap-Vulnerability-Skripte und OpenVAS ergänzen sich in Reichweite und Tiefe.
- False Positives erfordern manuelle Prüfung jedes automatisierten Fundes.
- Kritische Systeme wie DC01 verdienen besondere Vorsicht bei automatisierten Scans.

9. Kontrollfragen

1. Was unterscheidet einen authentifizierten von einem nicht authentifizierten Scan? (leicht)
2. Was ist ein False Positive? (leicht)
3. Warum sollte ein Vulnerability-Scan nicht zuerst gegen DC01 erfolgen? (mittel)

4. Welches Nmap-Skript-Kategorie prüft gezielt auf bekannte Schwachstellen? (mittel)
5. Ein automatisierter Scan meldet eine kritische Schwachstelle, die manuelle Prüfung zeigt aber, dass der betroffene Dienst gar nicht aktiv ist. Wie gehst du mit diesem Widerspruch um? (schwer)

Lösungshinweise: 1. Ein authentifizierter Scan meldet sich mit Zugangsdaten an und findet dadurch mehr als ein nicht authentifizierter. 2. Ein gemeldeter Fund, der sich bei näherer Prüfung als nicht tatsächlich zutreffend herausstellt. 3. Weil ein unerwarteter Ausfall eines Domain Controllers die gesamte Domäne beeinträchtigen würde. 4. `--script=vuln`. 5. Den Fund als False Positive dokumentieren, mit Begründung (Dienst nicht aktiv), statt ihn unreflektiert in den Bericht zu übernehmen oder ihn ebenso unreflektiert zu verwerfen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen nicht authentifizierten Scan durchgeführt hast,
- [ ] einen authentifizierten Scan vorbereitet und den Unterschied bewertet hast,
- [ ] einen Fund gegen eine öffentliche CVE-Quelle geprüft hast,
- [ ] einen False Positive erkennen und korrekt dokumentieren kannst.

Kapitel 13: Grundlagen von Metasploit

Lernziele

- Du nutzt die Metasploit-Datenbank zur Verwaltung von Scan-Ergebnissen gegen Windows-Ziele.
- Du führst ein Auxiliary-Modul gegen SMB auf CLIENT02 aus.
- Du setzt einen passenden, dokumentierten Exploit gegen CLIENT02 ein.

1. Einführung

Nmap und die vorherigen Analysewerkzeuge liefern Informationen. Metasploit bündelt tausende Module, um aus diesen Informationen gezielt einen Zugang zu erlangen.

2. Technische Grundlagen

Metasploit organisiert Funde in Workspaces, gestützt auf eine lokale Datenbank, identisch zum Vorgehen in der Linux-Schwesterreihe.

Beispiel 13.1: Workspace anlegen und Nmap-Scan einbinden

```
$ msfconsole -q
msf6 > workspace -a winsec-band3
msf6 > db_nmap -sV 192.168.56.60
```

3. Fachbegriffe

Ein **Auxiliary-Modul** führt Hilfsaufgaben wie Scanning oder Login-Versuche ohne eigenen Exploit aus. Ein **Exploit-Modul** nutzt eine konkrete Schwachstelle, ein **Payload** läuft danach auf dem Zielsystem, bei Windows häufig als **Meterpreter**-Sitzung.

4. Praxis und Konfiguration

Beispiel 13.2: SMB-Signierung-Status per Auxiliary prüfen

```
msf6 > use auxiliary/scanner/smb/smb_version
msf6 auxiliary(smb_version) > set RHOSTS 192.168.56.60
msf6 auxiliary(smb_version) > run
```

Beispiel 13.3: Einen passenden Exploit suchen und einsetzen (Konzept)

```
msf6 > search type:exploit platform:windows <erkannter Dienst>
msf6 > use <gefundenes Modul>
msf6 exploit(...) > set RHOSTS 192.168.56.60
msf6 exploit(...) > check
msf6 exploit(...) > run
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Setze Exploit-Module ausschließlich gegen CLIENT02 in diesem isolierten Labor ein. `check` vor `run` nutzen, wo verfügbar, um Fehlalarme zu vermeiden, bevor ein invasiverer Schritt folgt.

6. Praktische Laborübungen

Übung A: Workspace und Auxiliary-Scan einrichten

Aufgabenstellung: Richte einen Workspace ein und prüfe CLIENT02 auf SMB-Details. **Ausgangssituation:** CLIENT02 nach Vorwort. **Voraussetzungen:** Metasploit mit aktiver Datenbank. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 13.1, 13.2. **Erwartetes Ergebnis:** SMB-Version und Betriebssysteminformationen von CLIENT02. **Überprüfung:** `hosts` und `services` zeigen CLIENT02 mit dem geprüften Dienst. **Fehleranalyse:** Keine Datenbankverbindung? `msfdb init` erneut ausführen. **Musterlösung:** Ein dokumentiertes Ergebnis des Auxiliary-Laufs im Workspace.

Übung B: Einen passenden Exploit gegen CLIENT02 einsetzen

Aufgabenstellung: Finde anhand der bisherigen Scan-Ergebnisse (Kapitel 4, 12) ein passendes Exploit-Modul und setze es gegen CLIENT02 ein. **Ausgangssituation:** Ergebnis aus Übung A sowie Kapitel 4, 12. **Voraussetzungen:** ein auf CLIENT02 tatsächlich vorhandener, bekanntermaßen verwundbarer Dienst oder eine Fehlkonfiguration (z. B. eine absichtlich veraltete Testanwendung). **Schritt-für-Schritt-Anleitung:** siehe Beispiel 13.3. **Erwartetes Ergebnis:** Eine Meterpreter-Sitzung mit Zugriff auf CLIENT02. **Überprüfung:** `sessions -l` zeigt die aktive Sitzung. **Fehleranalyse:** `check` meldet „not vulnerable“? Versionsstand des Dienstes gegenprüfen. **Musterlösung:** Eine dokumentierte, erfolgreiche Sitzung mit Angabe von Modul, Zielsystem und erreichtem Rechteniveau.

7. Typische Fehler

Fehler	Ursache	Behebung
Kein Workspace angelegt, Ergebnisse verstreut	Datenbankfunktion nicht genutzt	immer zuerst Workspace anlegen
Exploit ohne vorheriges `check` direkt ausgeführt	Ungeduld	`check` nutzen, wo verfügbar
Modul gegen falsche IP-Adresse gerichtet	Verwechslung der Laborsysteme	Zieladresse vor jedem Lauf prüfen

8. Zusammenfassung

Zusammenfassung

- Metasploit bündelt Scanning, Exploitation und Post-Exploitation mit Datenbank-Unterstützung.
- Auxiliary-Module prüfen ohne eigenen Exploit, Exploit-Module nutzen konkrete Schwachstellen aktiv aus.
- Meterpreter ist ein verbreiteter Payload speziell für Windows-Ziele.
- Jeder Einsatz bleibt auf CLIENT02 im eigenen Labor beschränkt.

9. Kontrollfragen

1. Wozu dient ein Workspace in Metasploit? (leicht)

2. Was ist Meterpreter? (leicht)
3. Wozu dient der Befehl `check`? (mittel)
4. Welcher Befehl bindet einen Nmap-Scan direkt in die Datenbank ein? (mittel)
5. Ein Exploit-Modul meldet nach `check` „target is not exploitable“. Welche zwei möglichen Ursachen prüfst du zuerst? (schwer)

Lösungshinweise: 1. Er trennt Hosts, Dienste und Funde verschiedener Projekte innerhalb derselben Installation. 2. Ein verbreiteter, speziell für Windows-Ziele entwickelter Payload mit erweiterten Post-Exploitation-Funktionen. 3. Er prüft Verwundbarkeit, ohne den eigentlichen Exploit auszulösen. 4. `db_nmap`. 5. Erstens, ob die tatsächlich installierte Version des Zieldienstes von der im Modul vorausgesetzten Version abweicht; zweitens, ob die Zieladresse korrekt gesetzt ist.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen Workspace angelegt und einen Nmap-Scan eingebunden hast,
- [ ] ein Auxiliary-Modul erfolgreich eingesetzt hast,
- [ ] einen passenden Exploit erfolgreich gegen CLIENT02 eingesetzt hast,
- [ ] den Unterschied zwischen Auxiliary- und Exploit-Modulen erklären kannst.

Kapitel 14: PowerShell Security

Lernziele

- Du aktivierst PowerShell-Skriptblockprotokollierung.
- Du erklärst den Zweck der Execution Policy und ihre Grenzen.
- Du erkennst ein verschleiertes (obfuskiertes) PowerShell-Kommando.

1. Einführung

PowerShell ist zugleich das wichtigste Verwaltungswerkzeug dieser Reihe und eines der am häufigsten für Angriffe missbrauchten Windows-Bordmittel, weil es standardmäßig vorhanden ist und direkten Systemzugriff erlaubt.

2. Technische Grundlagen

Die **Execution Policy** steuert, welche PowerShell-Skripte ausgeführt werden dürfen, ist aber ausdrücklich keine Sicherheitsgrenze: Sie lässt sich mit einfachen, dokumentierten Mitteln umgehen (z. B. durch direktes Einfügen von Code statt Ausführen einer Datei) und schützt vor allem gegen versehentliche Skriptausführung, nicht vor vorsätzlichem Missbrauch.

Beispiel 14.1: Skriptblockprotokollierung aktivieren

```
Computerkonfiguration -> Richtlinien -> Administrative Vorlagen -> Windows-Komponenten ->
Windows PowerShell -> "Protokollierung von PowerShell-Skriptblöcken aktivieren" -> Aktiviert
```

3. Fachbegriffe

Obfuskation verschleiert eine Befehlszeile (etwa durch Base64-Kodierung oder Zeichenersetzung), um einer einfachen, musterbasierten Erkennung zu entgehen. **AMSI** (Antimalware Scan Interface) erlaubt Defender und anderen Sicherheitsprodukten, auch dynamisch erzeugten PowerShell-Code zur Laufzeit zu prüfen, bevor er ausgeführt wird.

4. Praxis und Konfiguration

Beispiel 14.2: Protokollierte Skriptblöcke auswerten

```
PS> Get-WinEvent -LogName "Microsoft-Windows-PowerShell/Operational" |
    Where-Object { $_.Id -eq 4104 } | Select-Object -First 10
```

Beispiel 14.3: Eine Base64-kodierte Befehlszeile dekodieren (zur Analyse, nicht zur Ausführung)

```
PS> [System.Text.Encoding]::Unicode.GetString(`
    [System.Convert]::FromBase64String("<kodierter Text>"))
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Dekodiere verdächtige Befehlszeilen ausschließlich zur Analyse, niemals zur versehentlichen Ausführung. Eine dekodierte, potenziell schädliche Befehlszeile bleibt gefährlich, auch wenn sie nur zur Untersuchung betrachtet wird.

6. Praktische Laborübungen

Übung A: Skriptblockprotokollierung aktivieren und auswerten

Aufgabenstellung: Aktiviere die Protokollierung per GPO und werte protokollierte Skriptblöcke nach Ausführung eines Testskripts aus. **Ausgangssituation:** CLIENT01 in der Domäne. **Voraussetzungen:** administrative Domänenrechte. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 14.1, danach ein Testskript ausführen, dann Beispiel 14.2. **Erwartetes Ergebnis:** Event-ID 4104 zeigt den vollständigen Inhalt des ausgeführten Skripts. **Überprüfung:** Der protokollierte Inhalt stimmt mit dem tatsächlich ausgeführten Skript überein. **Fehleranalyse:** Keine Protokollierung? `gpupdate /force` ausführen und Richtlinienanwendung prüfen. **Musterlösung:** Eine dokumentierte, vollständige Skriptblockprotokollierung.

Übung B: Eine obfuskierte Befehlszeile analysieren

Aufgabenstellung: Erzeuge testweise eine Base64-kodierte PowerShell-Befehlszeile und dekodiere sie zur Analyse. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** Eine harmlose Testbefehlszeile kodieren (`-EncodedCommand`), danach mit Beispiel 14.3 dekodieren, ohne sie auszuführen. **Erwartetes Ergebnis:** Der ursprüngliche, lesbare Befehl erscheint nach der Dekodierung. **Überprüfung:** Der dekodierte Text stimmt mit dem ursprünglich kodierten Befehl überein. **Fehleranalyse:** Fehlerhafte Dekodierung? Zeichenkodierung (`Unicode` vs. `UTF8`) prüfen, PowerShell nutzt standardmäßig UTF-16LE für `-EncodedCommand`. **Musterlösung:** Eine korrekt dekodierte, analysierte Testbefehlszeile.

7. Typische Fehler

Fehler	Ursache	Behebung
Execution Policy als vollständigen Schutz missverstanden	Begriff falsch eingeordnet	sie ist keine Sicherheitsgrenze, nur Schutz vor Versehen
Skriptblockprotokollierung nie aktiviert	fehlendes Bewusstsein für ihren Wert	als Standardmaßnahme in jeder Domäne einrichten
Verdächtige Befehlszeile versehentlich ausgeführt statt nur analysiert	Unachtsamkeit	Dekodierung strikt von Ausführung trennen

8. Zusammenfassung

Zusammenfassung

- Die Execution Policy ist keine Sicherheitsgrenze, sondern Schutz vor versehentlicher Ausführung.
- Skriptblockprotokollierung macht auch dynamisch erzeugten PowerShell-Code nachvollziehbar.
- AMSI erlaubt Sicherheitsprodukten, PowerShell-Code zur Laufzeit zu prüfen.
- Obfuskierte Befehlszeilen sollten ausschließlich zur Analyse, nie zur Ausführung dekodiert werden.

9. Kontrollfragen

1. Ist die Execution Policy eine Sicherheitsgrenze? (leicht)
2. Welche Event-ID protokolliert PowerShell-Skriptblöcke? (leicht)
3. Was ist Obfuskation? (mittel)
4. Wozu dient AMSI? (mittel)
5. Ein Administrator vertraut ausschließlich auf die Execution Policy, um unerwünschte PowerShell-Ausführung zu verhindern. Welches konkrete Risiko bleibt bestehen? (schwer)

Lösungshinweise: 1. Nein, sie schützt vor allem vor versehentlicher, nicht vor vorsätzlicher Ausführung. 2. 4104. 3. Die Verschleierung einer Befehlszeile, um einfacher, musterbasierter Erkennung zu entgehen. 4. Es erlaubt Sicherheitsprodukten, auch dynamisch erzeugten PowerShell-Code zur Laufzeit zu prüfen. 5. Direkt eingefügter Code, der nicht als Datei ausgeführt wird, oder eine `-EncodedCommand`-Befehlszeile umgeht die Execution Policy mit dokumentierten, einfachen Mitteln, sodass ein entschlossener Angreifer sie leicht umgeht.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] Skriptblockprotokollierung aktiviert und ausgewertet hast,
- [ ] eine obfuskierte Befehlszeile sicher dekodiert hast,
- [ ] erklären kannst, warum die Execution Policy keine Sicherheitsgrenze ist,
- [ ] den Zweck von AMSI erklären kannst.

Kapitel 15: Malware- und Rootkit-Analyse

Lernziele

- Du ordnest statische und dynamische Malware-Analyse voneinander ab.
- Du analysierst eine verdächtige Datei in einer isolierten Umgebung.
- Du erkennst typische Persistenzmechanismen nach einer erfolgreichen Kompromittierung.

1. Einführung

Nach einer erfolgreichen Rechteausweitung (Kapitel 7) oder Exploitation (Kapitel 13) sichert sich ein tatsächlicher Angreifer häufig dauerhaften Zugriff. Dieses Kapitel zeigt, woran sich das erkennen lässt, und wie verdächtige Dateien sicher untersucht werden.

2. Technische Grundlagen

Statische Analyse untersucht eine Datei, ohne sie auszuführen (Strings, Metadaten, digitale Signatur). **Dynamische Analyse** führt sie in einer isolierten Umgebung aus und beobachtet ihr tatsächliches Verhalten (Netzwerkverbindungen, Dateiänderungen, Registry-Einträge).

Beispiel 15.1: Statische Analyse mit Bordmitteln

```
PS> Get-AuthenticodeSignature -FilePath verdaechtig.exe
PS> Get-FileHash -FilePath verdaechtig.exe -Algorithm SHA256
```

3. Fachbegriffe

Eine **Sandbox** ist eine isolierte Umgebung, in der verdächtige Dateien gefahrlos ausgeführt und beobachtet werden können, ohne das eigentliche System zu gefährden. **Persistenz** bezeichnet Mechanismen, mit denen sich ein Angreifer nach einem Neustart erneut Zugriff verschafft, etwa über einen manipulierten Autostart-Eintrag (Kapitel 6) oder eine geplante Aufgabe.

4. Praxis und Konfiguration

Beispiel 15.2: Datei-Hash gegen eine öffentliche Datenbank prüfen (Konzept)

```
Den mit Get-FileHash ermittelten SHA256-Wert bei einem öffentlichen
Analysedienst (z. B. VirusTotal) nachschlagen, ohne die Datei selbst
hochzuladen, falls sie vertrauliche Informationen enthalten könnte.
```

Beispiel 15.3: Persistenzmechanismen prüfen (Wiederholung aus Kapitel 6, vertieft)

```
PS> Get-ScheduledTask | Where-Object { $_.State -eq "Ready" } |
    Select-Object TaskName, TaskPath
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Führe verdächtige, potenziell schädliche Dateien ausschließlich in einer dafür vorgesehenen, vollständig isolierten Sandbox-VM aus, ohne Netzwerkverbindung zu den übrigen vier Laborsystemen. Eine echte, unkontrolliert ausgeführte Schadsoftware kann sich sonst im gesamten Labor ausbreiten.

6. Praktische Laborübungen

Übung A: Statische Analyse einer Testdatei

Aufgabenstellung: Führe eine statische Analyse einer harmlosen Testdatei durch (z. B. ein unsigniertes, selbst erstelltes Skript). **Ausgangssituation:** CLIENT01, eine selbst erstellte Testdatei ohne digitale Signatur. **Voraussetzungen:** keine besonderen. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 15.1. **Erwartetes Ergebnis:** Die Datei zeigt keine gültige Signatur und einen eindeutigen Hash-Wert. **Überprüfung:** Der Hash-Wert lässt sich reproduzierbar erneut berechnen. **Fehleranalyse:** Unerwartete Signatur? Prüfen, ob versehentlich eine andere, signierte Datei analysiert wurde. **Musterlösung:** Eine vollständige, dokumentierte statische Analyse.

Übung B: Persistenzmechanismen nach einer vorherigen Übung prüfen

Aufgabenstellung: Prüfe CLIENT02 nach den Übungen aus Kapitel 7 und 13 auf eigene, versehentlich hinterlassene Spuren. **Ausgangssituation:** CLIENT02 nach den genannten Kapiteln. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 15.3, zusätzlich Autoruns (Kapitel 6) erneut ausführen. **Erwartetes Ergebnis:** Eine vollständige Liste aller während der Übungen vorgenommenen Änderungen. **Überprüfung:** Jede gefundene Änderung lässt sich einer konkreten, eigenen Übung zuordnen. **Fehleranalyse:** Unerklärliche Einträge gefunden? Snapshot-Historie der VM prüfen. **Musterlösung:** Eine vollständige Übersicht eigener Testspuren.

7. Typische Fehler

Fehler	Ursache	Behebung
Verdächtige Datei ohne Isolation ausgeführt	fehlendes Bewusstsein für Ausbreitungsrisiko	ausschließlich in isolierter Sandbox-VM ausführen
Datei mit vertraulichem Inhalt bei Online-Dienst hochgeladen	Datenschutz nicht bedacht	zunächst nur den Hash-Wert prüfen
Eigene Testspuren mit echtem Angriff verwechselt	fehlende Dokumentation eigener Übungen	jede eigene Änderung während der Übungen protokollieren

8. Zusammenfassung

Zusammenfassung

- Statische Analyse untersucht eine Datei ohne Ausführung, dynamische beobachtet tatsächliches Verhalten.
- Eine Sandbox isoliert die Ausführung verdächtiger Dateien vom restlichen Labor.
- Persistenzmechanismen wie geplante Aufgaben oder Autostart-Einträge sind klassische Indikatoren.
- Verdächtige Dateien gehören niemals ohne vollständige Netzwerkisolation ausgeführt.

9. Kontrollfragen

1. Was unterscheidet statische von dynamischer Malware-Analyse? (leicht)
2. Was ist eine Sandbox? (leicht)
3. Warum sollte eine Datei mit vertraulichem Inhalt nicht direkt bei einem Online-Dienst hochgeladen werden? (mittel)
4. Nenne zwei manuell prüfbare Persistenzindikatoren unter Windows. (mittel)
5. Warum sollte selbst im eigenen, isolierten Labor eine verdächtige Datei nicht einfach auf einem der Haupttestsysteme ausgeführt werden? (schwer)

Lösungshinweise: 1. Statische Analyse untersucht die Datei ohne Ausführung, dynamische beobachtet ihr tatsächliches Verhalten bei der Ausführung. 2. Eine isolierte Umgebung zur gefahrlosen Ausführung und Beobachtung verdächtiger Dateien. 3. Weil der Hochladedienst die Datei selbst einsehen und speichern könnte. 4. Geplante Aufgaben und Autostart-Einträge. 5. Echte Schadsoftware kann sich unkontrolliert über das Netzwerk auf die anderen vier Laborsysteme ausbreiten; eine vollständig isolierte, separate Sandbox-VM ohne Netzwerkverbindung verhindert das.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] eine statische Analyse einer Testdatei durchgeführt hast,
- [ ] Persistenzmechanismen systematisch geprüft hast,
- [ ] den Unterschied zwischen statischer und dynamischer Analyse erklären kannst,
- [ ] erklären kannst, warum eine Sandbox vollständig isoliert sein muss.

Kapitel 16: Windows Event Forensics

Lernziele

- Du rekonstruierst einen Anmeldeverlauf aus mehreren Event-IDs.
- Du erkennst Spuren lateraler Bewegung im Sicherheitsprotokoll.
- Du korrelierst Ereignisse zwischen mehreren Systemen.

1. Einführung

Band 2, Kapitel 14 hat Protokollierung und Sammlung eingerichtet. Dieses Kapitel nutzt die gesammelten Daten gezielt für eine rückblickende Untersuchung, wie sie nach einem echten oder simulierten Vorfall nötig wird.

2. Technische Grundlagen

Ein vollständiger Anmeldeverlauf setzt sich aus mehreren Event-IDs zusammen: **4624** (erfolgreiche Anmeldung), **4634** (Abmeldung), **4672** (Zuweisung besonderer Rechte, typisch bei administrativer Anmeldung), **4648** (Anmeldung mit explizit anderen Zugangsdaten).

Beispiel 16.1: Anmeldeverlauf für ein bestimmtes Konto rekonstruieren

```
PS> Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624,4634} |
    Where-Object { $_.Message -match "testuser2" }
```

3. Fachbegriffe

Lateral Movement (Kapitel 2) hinterlässt typische Spuren: Event-ID 4648 auf dem Quellsystem (explizite Zugangsdaten verwendet), kombiniert mit 4624 auf dem Zielsystem zur selben Zeit. Die **Logon Type** in Event-ID 4624 unterscheidet unter anderem interaktive (2), Netzwerk- (3) und RDP-Anmeldungen (10).

4. Praxis und Konfiguration

Beispiel 16.2: Logon-Type eines Anmeldeereignisses prüfen

```
PS> Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624} -MaxEvents 1 |
    Select-Object -ExpandProperty Message
```

Beispiel 16.3: Ereignisse zwischen zwei Systemen korrelieren (Konzept)

```
Auf SRV01: Event-ID 4624, Logon Type 3, Quelle CLIENT02, Zeitpunkt T
Auf CLIENT02: Event-ID 4648 kurz vor Zeitpunkt T, mit denselben Zugangsdaten
-> deutet auf eine explizite, laterale Bewegung von CLIENT02 zu SRV01 hin
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Eine einzelne Event-ID allein erlaubt selten eine sichere Aussage. Erst die Korrelation mehrerer Ereignisse über Zeit und Systeme hinweg ergibt ein verlässliches Bild – ein Grund, warum zentrale Sammlung (Band 2, Kapitel 14) so wichtig ist.

6. Praktische Laborübungen

Übung A: Anmeldeverlauf eines Testkontos rekonstruieren

Aufgabenstellung: Rekonstruiere den vollständigen An- und Abmeldeverlauf eines Testkontos über einen Testzeitraum. **Ausgangssituation:** Testkonto mit mehreren An- und Abmeldungen auf CLIENT01. **Voraussetzungen:** Zugriff auf das Sicherheitsprotokoll. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 16.1. **Erwartetes Ergebnis:** Eine chronologische Liste aller An- und Abmeldungen. **Überprüfung:** Die Anzahl der Ereignisse stimmt mit den tatsächlichen Anmeldevorgängen überein. **Fehleranalyse:** Lücken im Verlauf? Prüfen, ob die Protokollgröße (Band 1, Kapitel 11) ältere Einträge bereits überschrieben hat. **Musterlösung:** Eine vollständige, chronologische Anmeldehistorie.

Übung B: Laterale Bewegung simulieren und im Protokoll erkennen

Aufgabenstellung: Melde dich von CLIENT02 mit expliziten, anderen Zugangsdaten auf SRV01 an und finde die Spuren in beiden Protokollen. **Ausgangssituation:** Ergebnis aus Übung A, Event Forwarding aus Band 2, Kapitel 14 aktiv. **Voraussetzungen:** ein Testkonto mit Zugriff auf beide Systeme. **Schritt-für-Schritt-Anleitung:** `runas /user:nova\testuser2 cmd` auf CLIENT02 ausführen, danach eine Verbindung zu SRV01 aufbauen, anschließend Beispiel 16.3 auf beiden Protokollen nachvollziehen. **Erwartetes Ergebnis:** Event-ID 4648 auf CLIENT02, Event-ID 4624 (Logon Type 3) auf SRV01, zeitlich korrelierbar. **Überprüfung:** Die Zeitstempel beider Ereignisse liegen plausibel nah beieinander. **Fehleranalyse:** Keine Korrelation erkennbar? Zeitsynchronisation zwischen den Systemen prüfen (Kerberos-Voraussetzung, Band 2, Kapitel 7). **Musterlösung:** Eine dokumentierte, korrelierte Rekonstruktion der lateralen Bewegung.

7. Typische Fehler

Fehler	Ursache	Behebung
Aussage auf Basis einer einzelnen Event-ID getroffen	fehlende Korrelation	immer mehrere Ereignisse und Systeme einbeziehen
Logon Type ignoriert	Detail als unwichtig eingestuft	Logon Type liefert wichtigen Kontext (interaktiv, Netzwerk, RDP)
Zeitstempel zwischen Systemen nicht verglichen	fehlende Zeitsynchronisation übersehen	Systemzeiten vor der Analyse auf Konsistenz prüfen

8. Zusammenfassung

Zusammenfassung

- Ein vollständiger Anmeldeverlauf setzt sich aus mehreren, zusammengehörigen Event-IDs zusammen.
- Logon Type unterscheidet interaktive, Netzwerk- und RDP-Anmeldungen.
- Laterale Bewegung hinterlässt korrelierbare Spuren auf Quell- und Zielsystem.
- Eine einzelne Event-ID erlaubt selten eine sichere Aussage ohne Korrelation.

9. Kontrollfragen

1. Welche Event-ID kennzeichnet eine erfolgreiche Anmeldung? (leicht)
2. Was unterscheidet Logon Type 2 von Logon Type 3? (leicht)
3. Warum braucht es mehrere Ereignisse für eine verlässliche Aussage? (mittel)
4. Welche zwei Event-IDs deuten gemeinsam auf laterale Bewegung hin? (mittel)
5. Zwei Systeme zeigen scheinbar korrelierende Anmeldeereignisse, aber mit einer Zeitabweichung von zwei Stunden. Was prüfst du als Erstes? (schwer)

Lösungshinweise: 1. 4624. 2. Logon Type 2 ist eine interaktive Anmeldung, Logon Type 3 eine Netzwerkanmeldung. 3. Weil eine einzelne Event-ID oft mehrdeutig ist und erst im Zusammenhang mit anderen Ereignissen ein verlässliches Bild ergibt. 4. 4648 (explizite Zugangsdaten) auf dem Quell- und 4624 auf dem Zielsystem. 5. Die Zeitsynchronisation (NTP) zwischen beiden Systemen, da eine Zeitabweichung die Korrelation verfälschen oder eine tatsächliche Korrelation unentdeckt lassen kann.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen vollständigen Anmeldeverlauf rekonstruiert hast,
- [ ] laterale Bewegung simuliert und im Protokoll erkannt hast,
- [ ] Logon Types unterscheiden kannst,
- [ ] erklären kannst, warum Zeitsynchronisation für die Korrelation wichtig ist.

Kapitel 17: Incident Response

Lernziele

- Du durchläufst die sechs klassischen Incident-Response-Phasen an einem simulierten AD-Vorfall.
- Du isolierst ein kompromittiertes System netzwerkseitig, ohne es auszuschalten.
- Du erstellst ein Playbook für einen Domain-Admin-Kompromittierungsverdacht.

1. Einführung

Alle vorherigen Kapitel haben Schwachstellen gefunden und ausgenutzt. Dieses Kapitel simuliert die Gegenseite: Wie reagiert man, wenn ein solcher Vorfall tatsächlich (simuliert) in einer Active-Directory-Umgebung eintritt?

2. Technische Grundlagen

Die sechs Phasen Preparation, Identification, Containment, Eradication, Recovery und Lessons Learned strukturieren jede Reaktion, unabhängig von der Vorfallart. In einer Domäne ist **Containment** oft komplexer als bei einem Einzelsystem: Ein kompromittiertes Konto zu sperren reicht unter Umständen nicht, wenn bereits weitere Konten oder Systeme betroffen sein könnten.

Beispiel 17.1: Phasenablauf am simulierten Vorfall (Konzept)

```
Identification -> Sysmon-Alarm (Band 2, Kap. 15) zeigt ungewöhnlichen Prozessstart auf SRV01
Containment    -> SRV01 netzwerkseitig isolieren, betroffenes Konto deaktivieren
Eradication    -> Ursache klären (z. B. Kapitel 7/13 dieses Bandes als Ursprung)
Recovery       -> System bereinigen oder aus Backup wiederherstellen (Band 2, Kap. 18)
Lessons Learned -> Fehlkonfiguration beheben, Erkennungsregel ergänzen
```

3. Fachbegriffe

Netzwerkseitige Isolation trennt ein System vom übrigen Netzwerk (z. B. über eine gezielte Firewall-Regel oder VLAN-Umschaltung), ohne es auszuschalten – wichtig, um flüchtige Daten für die spätere Forensik (Kapitel 18) zu erhalten. Ein **Playbook** ist eine im Voraus festgelegte, kurze Anleitung für ein wiederkehrendes Szenario.

4. Praxis und Konfiguration

Beispiel 17.2: Ein System netzwerkseitig isolieren, ohne es abzuschalten

```
PS> New-NetFirewallRule -DisplayName "Incident-Isolation" -Direction Outbound `
    -Action Block -RemoteAddress Any
PS> New-NetFirewallRule -DisplayName "Incident-Isolation-In" -Direction Inbound `
    -Action Block -RemoteAddress Any
```

Beispiel 17.3: Ein einfaches Vorfall-Protokoll führen

```
14:02 - Sysmon-Alarm: ungewöhnlicher Prozessstart auf SRV01
14:04 - Bestätigt: Prozess stammt aus eigener Übung in Kapitel 13
14:05 - Keine weitere Maßnahme nötig, als eigener Testlauf eingestuft
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Isoliere ein verdächtiges System netzwerkseitig, bevor du es ausschaltest. Ein vorschnelles Abschalten zerstört flüchtige Daten (Arbeitsspeicher, aktive Verbindungen), die für die spätere forensische Auswertung entscheidend sein können.

6. Praktische Laborübungen

Übung A: Einen simulierten Vorfall vollständig durchlaufen

Aufgabenstellung: Nutze die aus Kapitel 13 erlangte Sitzung auf CLIENT02 als simulierten Vorfall und durchlaufe alle sechs Phasen. **Ausgangssituation:** Ergebnis aus Kapitel 13, Sysmon aus Band 2 aktiv. **Voraussetzungen:** Zugriff auf CLIENT02 als Administrator zur Reaktion.
Schritt-für-Schritt-Anleitung: siehe Beispiel 17.1, jede Phase schriftlich dokumentieren. **Erwartetes Ergebnis:** Eine vollständige, chronologische Dokumentation aller sechs Phasen. **Überprüfung:** Jede Phase ist klar von der vorherigen abgegrenzt und begründet. **Fehleranalyse:** Phasen vermischt? Containment von Eradication sauber trennen. **Musterlösung:** Eine vollständige, nachvollziehbare Dokumentation von der Erkennung bis zur Lehre für die Zukunft.

Übung B: Netzwerkseitige Isolation und Playbook für Domain-Admin-Verdacht

Aufgabenstellung: Isoliere CLIENT02 nach Beispiel 17.2 und erstelle ein Playbook für den Verdachtsfall einer Domain-Admin-Kompromittierung. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** administrative Rechte auf CLIENT02. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 17.2, danach ein fünfstufiges Playbook analog zum Muster aus Band 2, Kapitel 8 formulieren.
Erwartetes Ergebnis: Ein isoliertes, aber weiterhin laufendes System sowie ein kurzes, konkretes Playbook. **Überprüfung:** Das System bleibt erreichbar für lokale Analyse, aber ohne Netzwerkverbindung nach außen. **Fehleranalyse:** System vollständig unerreichbar? Prüfen, ob versehentlich auch lokale Verwaltungszugänge blockiert wurden. **Musterlösung:** Eine erfolgreiche Isolation und ein fünfstufiges, konkretes Playbook.

7. Typische Fehler

Fehler	Ursache	Behebung
System vorschnell ausgeschaltet	Reihenfolge nicht beachtet	immer zuerst isolieren, dann forensisch sichern (Kapitel 18)
Nur das eine kompromittierte Konto gesperrt	mögliche weitere betroffene Konten nicht geprüft	mit BloodHound (Kapitel 8) nach erreichbaren, verwandten Konten suchen
Playbook zu allgemein formuliert	fehlende Konkretisierung	auf das konkrete AD-Szenario zuschneiden

8. Zusammenfassung

Zusammenfassung

- Die sechs Incident-Response-Phasen gelten auch für AD-Vorfälle, mit komplexerem Containment.

- Netzwerkseitige Isolation erhält flüchtige Daten für die spätere Forensik.
- Ein Playbook für Domain-Admin-Verdacht sollte besonders kurz und klar sein, da hier höchste Priorität gilt.
- Lessons Learned sollte auch bei einem Fehlalarm durchlaufen werden.

9. Kontrollfragen

1. Nenne die sechs Incident-Response-Phasen. (leicht)
2. Warum sollte ein System isoliert statt ausgeschaltet werden? (leicht)
3. Warum ist Containment in einer Domäne komplexer als bei einem Einzelsystem? (mittel)
4. Was bewirkt netzwerkseitige Isolation im Unterschied zum Ausschalten? (mittel)
5. Ein Alarm stellt sich nach der Untersuchung als eigene, autorisierte Testaktivität heraus. Welche Phasen durchläufst du trotzdem noch, und warum? (schwer)

Lösungshinweise: 1. Preparation, Identification, Containment, Eradication, Recovery, Lessons Learned. 2. Weil ein vorschnelles Ausschalten flüchtige Daten für die Forensik zerstört. 3. Weil mehrere Konten oder Systeme betroffen sein könnten, nicht nur das eine, ursprünglich auffällige. 4. Es trennt das System vom Netzwerk, lässt es aber für lokale Analyse weiterlaufen. 5. Lessons Learned durchläufst du trotzdem, da sich daraus etwa die Erkenntnis ergeben kann, Testaktivitäten künftig vorab anzukündigen, um unnötige Alarmierung zu vermeiden.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen simulierten AD-Vorfall vollständig durch alle sechs Phasen begleitet hast,
- [ ] ein System netzwerkseitig isoliert hast, ohne es auszuschalten,
- [ ] ein Playbook für Domain-Admin-Verdacht erstellt hast,
- [ ] erklären kannst, warum Containment in einer Domäne komplexer ist.

Kapitel 18: Digitale Forensik unter Windows

Lernziele

- Du sicherst flüchtige Daten eines Windows-Systems in korrekter Reihenfolge.
- Du erstellst ein forensisches Abbild eines Datenträgers mit Prüfsumme.
- Du dokumentierst eine lückenlose Chain of Custody.

1. Einführung

Nach Eradication und Recovery (Kapitel 17) bleibt oft eine letzte Frage: Was ist genau passiert, und lässt sich das gerichtsfest belegen? Dieses Kapitel vertieft die forensische Nachbereitung unter Windows.

2. Technische Grundlagen

Forensische Sicherung folgt der **Order of Volatility**: flüchtige Daten zuerst (Arbeitsspeicher, aktive Netzwerkverbindungen), dauerhaftere Daten danach (Festplatteninhalt), da flüchtige Daten beim Abschalten unwiederbringlich verloren gehen.

Beispiel 18.1: Flüchtigen Zustand sichern (vor jeder Abschaltung)

```
PS> Get-Process | Export-Csv prozesse.csv
PS> Get-NetTCPConnection | Export-Csv verbindungen.csv
PS> Get-WinEvent -LogName Security -MaxEvents 1000 | Export-Csv sicherheitsereignisse.csv
```

3. Fachbegriffe

Ein **forensisches Abbild** (Image) ist eine vollständige, bitgenaue Kopie eines Datenträgers mit zugehöriger Prüfsumme zum Nachweis der Unveränderheit. Die **Chain of Custody** dokumentiert lückenlos, wer wann auf ein Beweismittel zugegriffen hat. **FTK Imager** ist ein verbreitetes, kostenloses Werkzeug zur forensischen Abbilderstellung unter Windows.

4. Praxis und Konfiguration

Beispiel 18.2: Ein forensisches Abbild mit FTK Imager erstellen (Konzept)

```
FTK Imager -> File -> Create Disk Image -> Physical Drive auswählen ->
Image Type: E01 -> Zielpfad angeben -> Verify after creating wird automatisch
eine Prüfsumme (MD5/SHA1) erzeugen und mit dem Original vergleichen
```

Beispiel 18.3: Arbeitsspeicher sichern (Konzept)

```
Werkzeuge wie WinPmem erstellen ein vollständiges RAM-Abbild für die
spätere Analyse mit Volatility, bevor das System ausgeschaltet wird.
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Arbeite forensisch ausschließlich mit Kopien, niemals direkt mit dem Originaldatenträger. Jede Veränderung am Original kann spätere Beweiskraft zerstören.

6. Praktische Laborübungen

Übung A: Flüchtige Daten in korrekter Reihenfolge sichern

Aufgabenstellung: Sichere den flüchtigen Zustand von CLIENT02 nach dem simulierten Vorfall aus Kapitel 17. **Ausgangssituation:** CLIENT02 noch im Zustand nach Kapitel 13/17, nicht neu gestartet. **Voraussetzungen:** administrative Rechte, bevor das System abgeschaltet wird. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 18.1, in der gezeigten Reihenfolge. **Erwartetes Ergebnis:** Drei Dateien mit Prozess-, Verbindungs- und Ereignisinformationen. **Überprüfung:** Die gesicherten Prozesse stimmen mit der zuvor über `sessions -l` beobachteten Aktivität überein. **Fehleranalyse:** System bereits neu gestartet? Dieser Schritt dokumentiert dann exemplarisch, warum die Reihenfolge entscheidend ist. **Musterlösung:** Drei vollständige, mit Zeitstempel versehene Sicherungsdateien.

Übung B: Ein forensisches Abbild mit Prüfsumme erstellen

Aufgabenstellung: Erstelle ein Abbild einer Testpartition oder eines Datei-Images und bestätige seine Integrität. **Ausgangssituation:** Eine zusätzliche Testpartition oder ein Datei-Image, keine produktiv genutzten Daten. **Voraussetzungen:** FTK Imager oder vergleichbares Werkzeug installiert. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 18.2. **Erwartetes Ergebnis:** Ein vollständiges Abbild mit automatisch erzeugter, verifizierter Prüfsumme. **Überprüfung:** Die Verifizierung nach der Erstellung meldet Übereinstimmung. **Fehleranalyse:** Verifizierung schlägt fehl? Erstellung wiederholen, möglicherweise auf einem anderen Zielmedium. **Musterlösung:** Ein verifiziertes, unverändertes forensisches Abbild samt Prüfsumme.

7. Typische Fehler

Fehler	Ursache	Behebung
Flüchtige Daten durch vorschnellen Neustart verloren	Order of Volatility nicht beachtet	immer flüchtige vor dauerhaften Daten sichern
Arbeit direkt am Originaldatenträger	fehlendes Bewusstsein für Beweiskraft	ausschließlich mit verifizierten Kopien arbeiten
Keine Prüfsumme erstellt oder verifiziert	Schritt übersprungen	Verifizierung immer unmittelbar nach dem Abbild durchführen

8. Zusammenfassung

Zusammenfassung

- Die Order of Volatility bestimmt, flüchtige vor dauerhaften Daten zu sichern.
- Ein forensisches Abbild ist eine bitgenaue Kopie mit zugehöriger, verifizierter Prüfsumme.
- Die Chain of Custody dokumentiert lückenlosen Zugriff auf Beweismittel.

- Forensische Arbeit erfolgt ausschließlich an Kopien, nie am Original.

9. Kontrollfragen

1. Was beschreibt die Order of Volatility? (leicht)
2. Welches Werkzeug wurde zur forensischen Abbilderstellung genannt? (leicht)
3. Warum sollte niemals direkt am Originaldatenträger gearbeitet werden? (mittel)
4. Was ist die Chain of Custody? (mittel)
5. Ein System muss aus betrieblichen Gründen sofort neu gestartet werden, bevor flüchtige Daten gesichert wurden. Welche Konsequenz hat das für eine spätere forensische Auswertung? (schwer)

Lösungshinweise: 1. Die Reihenfolge, in der Daten gesichert werden sollten, von flüchtig zu dauerhaft. 2. FTK Imager. 3. Weil jede Veränderung am Original die spätere Beweiskraft zerstören kann. 4. Die lückenlose Dokumentation, wer wann auf ein Beweismittel zugegriffen hat. 5. Flüchtige Daten wie Arbeitsspeicherinhalt und aktive Verbindungen gehen unwiederbringlich verloren, was die spätere Rekonstruktion des genauen Tathergangs erheblich erschwert oder unmöglich macht.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] flüchtige Daten in korrekter Reihenfolge gesichert hast,
- [ ] ein forensisches Abbild mit Prüfsumme erstellt und verifiziert hast,
- [ ] die Chain of Custody erklären kannst,
- [ ] die Konsequenz eines vorschnellen Neustarts einschätzen kannst.

Kapitel 19: Security Auditing

Lernziele

- Du planst ein vollständiges Audit mit klar definiertem Scope und Methodik für eine AD-Umgebung.
- Du führst ein strukturiertes Audit durch, das mehrere Kapitel dieses Bandes verbindet.
- Du bewertest Funde nach CVSS.

1. Einführung

Die vorherigen Kapitel haben einzelne Werkzeuge und Techniken behandelt. Ein Security Audit verbindet sie zu einem methodischen, vollständigen Prüfprozess für die gesamte Domäne.

2. Technische Grundlagen

Ein Audit durchläuft typischerweise Reconnaissance, Scanning, Schwachstellenanalyse, kontrollierte Ausnutzung und Bewertung, mit klar definiertem Scope (Kapitel 1).

Beispiel 19.1: Ablauf eines strukturierten AD-Audits (Konzept)

```
1. Scope festlegen (Kapitel 1)
2. Reconnaissance und Enumeration (Kapitel 3-6)
3. Privilege Escalation und AD-Angriffspfade (Kapitel 7-10)
4. Passwort- und Exploitation-Tests (Kapitel 11-13)
5. Schweregrad-Bewertung jedes Fundes
6. Bericht (Kapitel 20)
```

3. Fachbegriffe

CVSS bewertet eine Schwachstelle anhand mehrerer Faktoren zu einem Wert zwischen 0 und 10. Ein **Finding** ist ein einzelner, dokumentierter Befund mit Beschreibung, Nachweis und Bewertung.

4. Praxis und Konfiguration

Beispiel 19.2: Ein Finding strukturiert dokumentieren

```
Titel:        Kerberoastable Dienstkonto mit schwachem Passwort
Schweregrad:  Hoch (CVSS ca. 7.5)
Nachweis:     Hashcat-Cracking-Erfolg innerhalb weniger Minuten (Kapitel 9)
Empfehlung:   Passwort auf gMSA umstellen oder mindestens 25 Zeichen zufällig setzen
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Ein Audit ohne vorab definierten Scope wiederholt den in Kapitel 1 beschriebenen Fehler. Jedes Audit in diesem Buch bleibt auf die fünf Laborsysteme beschränkt.

6. Praktische Laborübungen

Übung A: Ein vollständiges Audit für die Domäne planen

Aufgabenstellung: Erstelle einen schriftlichen Audit-Plan nach Beispiel 19.1 für die gesamte Labor-Domäne. **Ausgangssituation:** Alle vorherigen Kapitel dieses Bandes abgeschlossen. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** Für jeden Schritt aus Beispiel 19.1 die konkret einzusetzenden Werkzeuge aus diesem Buch benennen. **Erwartetes Ergebnis:** Ein vollständiger, kapitelübergreifender Plan. **Überprüfung:** Jeder Schritt verweist auf ein konkretes, bereits in diesem Buch behandeltes Kapitel. **Fehleranalyse:** Lücke im Plan? Inhaltsverzeichnis dieses Bandes erneut konsultieren. **Musterlösung:** Ein vollständiger Plan mit konkreter Werkzeugzuordnung.

Übung B: Drei Findings mit CVSS-Einschätzung dokumentieren

Aufgabenstellung: Dokumentiere drei tatsächliche Funde aus den vorherigen Kapiteln nach dem Schema aus Beispiel 19.2. **Ausgangssituation:** Funde aus Kapitel 3-17 liegen vor. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** Drei unterschiedliche Funde auswählen, jeweils grob nach CVSS einschätzen. **Erwartetes Ergebnis:** Drei vollständige, strukturierte Findings. **Überprüfung:** Jeder Schweregrad ist nachvollziehbar begründet. **Fehleranalyse:** Einschätzung wirkt willkürlich? Mit einem CVSS-Rechner im Internet gegenprüfen. **Musterlösung:** Drei vollständige Findings mit Titel, Schweregrad, Nachweis und Empfehlung.

7. Typische Fehler

Fehler	Ursache	Behebung
Audit ohne Scope begonnen	Kapitel 1 nicht konsequent angewendet	Scope immer vor Beginn schriftlich festlegen
Findings ohne konkreten Nachweis	Behauptung statt Beleg	jedes Finding mit konkretem, nachvollziehbarem Beleg versehen
Schweregrad willkürlich vergeben	kein anerkanntes System genutzt	CVSS oder vergleichbares System konsequent anwenden

8. Zusammenfassung

Zusammenfassung

- Ein Security Audit verbindet die Einzelwerkzeuge dieses Bandes zu einem methodischen Prozess.
- CVSS bewertet Schwachstellen nach einem anerkannten, vergleichbaren System.
- Jedes Finding braucht Titel, Schweregrad, Nachweis und Empfehlung.
- Der Scope aus Kapitel 1 gilt für jedes Audit unverändert.

9. Kontrollfragen

1. Wofür steht CVSS? (leicht)
2. Was gehört in ein vollständiges Finding? (leicht)
3. Warum sollte jedes Finding einen konkreten Nachweis enthalten? (mittel)
4. In welcher Reihenfolge läuft ein strukturiertes Audit typischerweise ab? (mittel)

5. Zwei Findings haben denselben CVSS-Wert, aber unterschiedliche Auswirkung auf den konkreten Geschäftsbetrieb. Sollten sie gleich priorisiert werden? (schwer)

Lösungshinweise: 1. Common Vulnerability Scoring System. 2. Titel, Schweregrad, Nachweis, Empfehlung. 3. Damit der Fund für Dritte nachvollziehbar und überprüfbar bleibt. 4. Scope, Reconnaissance/Scanning, Schwachstellenanalyse, kontrollierte Ausnutzung, Bewertung, Bericht. 5. Nicht zwingend: CVSS bewertet die technische Schwere, der tatsächliche Geschäftskontext kann eine abweichende Priorisierung rechtfertigen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen vollständigen Audit-Plan erstellt hast,
- [ ] drei Findings mit CVSS-Einschätzung dokumentiert hast,
- [ ] den Unterschied zwischen technischer Schwere und Geschäftsauswirkung erklären kannst,
- [ ] ein Audit konsequent innerhalb eines definierten Scopes durchführst.

Kapitel 20: Professionelle Sicherheitsberichte

Lernziele

- Du gliederst einen Sicherheitsbericht nach anerkanntem Aufbau.
- Du schreibst eine Management Summary für ein nicht-technisches Publikum.
- Du erstellst einen vollständigen Abschlussbericht für das Audit aus Kapitel 19.

1. Einführung

Das beste Audit bleibt wirkungslos, wenn sein Ergebnis nicht verständlich und handlungsleitend kommuniziert wird. Dieses letzte Kapitel schließt den Bogen der gesamten dreibändigen Reihe.

2. Technische Grundlagen

Ein professioneller Sicherheitsbericht richtet sich an zwei Zielgruppen gleichzeitig: technisches Personal, das Findings im Detail nachvollziehen muss, und Entscheidungsträger, die eine knappe, verständliche Gesamteinschätzung brauchen.

Beispiel 20.1: Grundgliederung eines Sicherheitsberichts

```
1. Management Summary (ein bis zwei Seiten, nicht-technisch)
2. Scope und Methodik
3. Zusammenfassung der Findings nach Schweregrad
4. Detaillierte Findings (Kapitel 19, Beispiel 19.2)
5. Empfehlungen, priorisiert
6. Anhänge (Rohdaten, Scan-Ausgaben, BloodHound-Pfade)
```

3. Fachbegriffe

Eine **Management Summary** fasst die wichtigsten Ergebnisse und Risiken ohne technischen Fachjargon zusammen. Eine **Executive Recommendation** benennt konkrete, priorisierte nächste Schritte für Entscheidungsträger.

4. Praxis und Konfiguration

Beispiel 20.2: Eine Management Summary (Auszug, Konzept)

```
Im Rahmen des vereinbarten Audits wurden mehrere Schwachstellen mit
mittlerem bis hohem Risiko in der Active-Directory-Umgebung festgestellt,
darunter ein Angriffspfad von einem gewöhnlichen Benutzerkonto bis zu
administrativen Rechten über die gesamte Domäne. CLIENT01 zeigte durch
die vorab umgesetzte Härtung aus Band 2 deutlich weniger und weniger
schwerwiegende Befunde als CLIENT02. Eine priorisierte Liste konkreter
Maßnahmen findet sich in Abschnitt 5.
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Ein Sicherheitsbericht ist selbst ein sensibles Dokument: Er beschreibt im Detail, wie eine Domäne verwundbar ist, teilweise inklusive gefundener Zugangsdaten oder Hash-Werte. Er gehört verschlüsselt übertragen, Zugriff darauf streng begrenzt, und nach Projektabschluss gemäß vereinbarter Frist gelöscht.

6. Praktische Laborübungen

Übung A: Management Summary schreiben

Aufgabenstellung: Schreibe eine Management Summary für das Audit aus Kapitel 19. **Ausgangssituation:** Drei Findings aus Kapitel 19, Übung B. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 20.2 als Vorlage, mit den tatsächlichen eigenen Findings füllen. **Erwartetes Ergebnis:** Ein bis zwei Absätze, vollständig frei von technischem Fachjargon. **Überprüfung:** Eine fachfremde Person könnte die Kernaussage verstehen, ohne Rückfragen zu stellen. **Fehleranalyse:** Zu technisch geraten? Jeden Fachbegriff durch eine allgemeinverständliche Umschreibung ersetzen. **Musterlösung:** Eine kurze, verständliche Zusammenfassung mit grober Risikoeinschätzung.

Übung B: Vollständigen Abschlussbericht erstellen

Aufgabenstellung: Erstelle einen vollständigen Bericht nach der Gliederung aus Beispiel 20.1, der das gesamte Audit aus Kapitel 19 zusammenfasst. **Ausgangssituation:** Ergebnis aus Übung A sowie alle Funde dieses Bandes. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** Jeden Abschnitt aus Beispiel 20.1 mit den tatsächlichen Inhalten aus Kapitel 1-19 füllen. **Erwartetes Ergebnis:** Ein vollständiges, in sich stimmiges Berichtsdokument. **Überprüfung:** Der Bericht lässt sich ohne Rückgriff auf dieses Buch eigenständig lesen und verstehen. **Fehleranalyse:** Abschnitt unvollständig? Entsprechendes Kapitel dieses Bandes erneut konsultieren. **Musterlösung:** Ein vollständiger, professioneller Abschlussbericht als Abschluss der gesamten dreibändigen Windows-Security-Reihe.

7. Typische Fehler

Fehler	Ursache	Behebung
Management Summary zu technisch	Fachjargon unreflektiert übernommen	konsequent für fachfremdes Publikum schreiben
Bericht ohne Empfehlungen	nur Befunde, keine nächsten Schritte	jedem Finding eine konkrete Empfehlung zuordnen
Bericht mit Klartext-Zugangsdaten ungeschützt versendet	Vertraulichkeit unterschätzt	verschlüsselt übertragen, Zugriff eng begrenzen

8. Zusammenfassung

Zusammenfassung

- Ein Sicherheitsbericht bedient technisches und nicht-technisches Publikum gleichzeitig.
- Die Management Summary fasst Ergebnisse verständlich und ohne Fachjargon zusammen.
- Jedes Finding braucht eine konkrete, priorisierte Empfehlung.
- Der Bericht selbst ist ein sensibles Dokument und verdient entsprechenden Schutz.

9. Kontrollfragen

1. An welche zwei Zielgruppen richtet sich ein Sicherheitsbericht typischerweise? (leicht)
2. Was gehört in eine Management Summary? (leicht)
3. Warum sollte eine Management Summary frei von Fachjargon sein? (mittel)
4. Warum ist ein Sicherheitsbericht selbst ein schützenswertes Dokument? (mittel)
5. Ein Auftraggeber möchte den Bericht nur als kurze E-Mail statt als vollständiges Dokument. Wie reagierst du professionell? (schwer)

Lösungshinweise: 1. Technisches Personal und nicht-technische Entscheidungsträger. 2. Eine kurze, verständliche Zusammenfassung der wichtigsten Ergebnisse und eine grobe Risikoeinschätzung. 3. Damit auch Entscheidungsträger ohne technischen Hintergrund die Kernaussage verstehen. 4. Weil er im Detail beschreibt, wie eine Umgebung verwundbar ist, teils inklusive gefundener Zugangsdaten. 5. Eine kurze Management Summary als E-Mail anbieten, aber auf den vollständigen, detaillierten Bericht als notwendige Grundlage für die technische Umsetzung der Empfehlungen bestehen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] eine verständliche Management Summary geschrieben hast,
- [ ] einen vollständigen, eigenständig lesbaren Abschlussbericht erstellt hast,
- [ ] erklären kannst, warum ein Bericht selbst schützenswert ist,
- [ ] auf eine unangemessene Verkürzungsanfrage professionell reagieren kannst.

Abschlussprojekt Band 3: Windows Security Audit

Führe ein vollständiges, eigenständiges Sicherheitsaudit über das gesamte Fünf-Maschinen-Labor durch:

1. **Informationssammlung:** Bedrohungsmodell (Kapitel 2), Netzwerkerkundung mit Nmap, AD-Enumeration (Kapitel 3-4).
2. **Active-Directory-Analyse:** BloodHound-Angriffspfad zu Domain Admin, Kerberoasting-Test (Kapitel 8-9).
3. **Netzwerkprüfung:** Wireshark-Mitschnitt mit Kerberos-/NTLM-Analyse (Kapitel 5, 10).
4. **Sicherheitsüberprüfung:** mindestens ein erfolgreicher Weg zur Rechteausweitung auf CLIENT02 (Kapitel 7) und ein erfolgreicher Metasploit-Einsatz (Kapitel 13).
5. **Schwachstellenbewertung:** CVSS-eingeschätzte Findings für alle gefundenen Schwachstellen (Kapitel 19).
6. **Risikobewertung und Handlungsempfehlungen:** priorisierte Liste mit Bezug auf die in Band 2 bereits umgesetzte Härtung von CLIENT01 und DC01.
7. **Abschließender Auditbericht:** ein vollständiger, zweiteiliger Bericht mit Management Summary und technischem Detailteil (Kapitel 20).

Dieses Projekt schließt die dreibändige Reihe: vom ersten, ungesicherten Client in Band 1 über die professionelle Härtung der gesamten Domäne in Band 2 bis zur eigenständigen, methodischen Überprüfung in Band 3.

Verzeichnis der Befehlsbeispiele

Beispiel V.1: Erweiterte Netzwerkarchitektur … Seite 9
Beispiel V.2: CLIENT02 einrichten … Seite 10
Beispiel V.3: KALI01 einrichten … Seite 10
Beispiel 1.1: Derselbe Angriff, drei rechtliche Einordnungen (Konzept) … Seite 11
Beispiel 1.2: Mindestinhalt eines AD-spezifischen Scope-Dokuments … Seite 11
Beispiel 2.1: Vertrauensgrenzen in der Labor-Domäne (Konzept) … Seite 14
Beispiel 2.2: STRIDE auf die Labor-Domäne angewendet (Auszug) … Seite 14
Beispiel 3.1: Domäneninformationen mit Bordmitteln sammeln … Seite 17
Beispiel 3.2: Mitglieder privilegierter Gruppen von einem eingeschränkten Konto aus ermitteln … Seite 17
Beispiel 3.3: Freigaben auf SRV01 ohne besondere Rechte auflisten … Seite 17
Beispiel 4.1: DC01 identifizieren … Seite 20
Beispiel 4.2: SMB-Details mit NSE auslesen … Seite 20
Beispiel 4.3: Vergleichsscan gegen beide Clients … Seite 20
Beispiel 5.1: Kerberos-Verkehr filtern … Seite 23
Beispiel 5.2: Unverschlüsselten LDAP-Verkehr auf Klartext-Zugangsdaten prüfen … Seite 23
Beispiel 5.3: NTLM-Fallback im Mitschnitt erkennen … Seite 23
Beispiel 6.1: Process Explorer mit Signaturprüfung starten … Seite 26
Beispiel 6.2: Autostart-Einträge mit Autoruns prüfen … Seite 26
Beispiel 6.3: Netzwerkverbindungen in Echtzeit beobachten … Seite 26
Beispiel 7.1: Dienstberechtigungen und Pfade prüfen … Seite 29
Beispiel 7.2: WinPEAS auf CLIENT02 ausführen … Seite 29
Beispiel 7.3: AlwaysInstallElevated prüfen … Seite 29
Beispiel 8.1: Daten mit SharpHound sammeln … Seite 32
Beispiel 8.2: Gesammelte Daten in BloodHound importieren und analysieren … Seite 32
Beispiel 9.1: Dienstkonten mit SPN auflisten … Seite 35
Beispiel 9.2: Ein Kerberos-Dienst-Ticket anfordern und offline speichern (nur eigenes Testkonto!) … Seite 35
Beispiel 9.3: Das gewonnene Ticket offline gegen eine Wortliste prüfen (Konzept, vergleiche John-the-Ripper-/Hashcat-Werkzeuge der Linux-Schwesterreihe) … Seite 36
Beispiel 10.1: Grundprinzip eines NTLM-Relay-Angriffs (Konzept) … Seite 38
Beispiel 10.2: LLMNR und NBT-NS deaktivieren (per GPO) … Seite 38
Beispiel 10.3: SMB-Signierung erzwingen (Wiederholung aus Band 2, Kapitel 8) … Seite 38
Beispiel 11.1: SAM-Datenbank-Struktur (Konzept, nicht direkt zugreifbar im laufenden Betrieb) … Seite 41
Beispiel 11.2: Lokale Hashes auf dem eigenen Testsystem mit administrativen Rechten extrahieren (nur eigenes Testkonto) … Seite 41
Beispiel 11.3: Geschwindigkeit von NTLM gegen ein Kerberos-Hash-Format vergleichen (vergleiche Linux-Schwesterreihe, Hashcat-Handbuch) … Seite 41
Beispiel 12.1: Nmap-Vulnerability-Skripte gegen CLIENT02 … Seite 44
Beispiel 12.2: Einen authentifizierten Scan mit OpenVAS vorbereiten (Konzept) … Seite 44
Beispiel 13.1: Workspace anlegen und Nmap-Scan einbinden … Seite 47
Beispiel 13.2: SMB-Signierung-Status per Auxiliary prüfen … Seite 47
Beispiel 13.3: Einen passenden Exploit suchen und einsetzen (Konzept) … Seite 47
Beispiel 14.1: Skriptblockprotokollierung aktivieren … Seite 50
Beispiel 14.2: Protokollierte Skriptblöcke auswerten … Seite 50
Beispiel 14.3: Eine Base64-kodierte Befehlszeile dekodieren (zur Analyse, nicht zur Ausführung) … Seite 50
Beispiel 15.1: Statische Analyse mit Bordmitteln … Seite 53
Beispiel 15.2: Datei-Hash gegen eine öffentliche Datenbank prüfen (Konzept) … Seite 53
Beispiel 15.3: Persistenzmechanismen prüfen (Wiederholung aus Kapitel 6, vertieft) … Seite 53
Beispiel 16.1: Anmeldeverlauf für ein bestimmtes Konto rekonstruieren … Seite 56
Beispiel 16.2: Logon-Type eines Anmeldeereignisses prüfen … Seite 56
Beispiel 16.3: Ereignisse zwischen zwei Systemen korrelieren (Konzept) … Seite 56
Beispiel 17.1: Phasenablauf am simulierten Vorfall (Konzept) … Seite 59
Beispiel 17.2: Ein System netzwerkseitig isolieren, ohne es abzuschalten … Seite 59

Beispiel 17.3: Ein einfaches Vorfall-Protokoll führen … Seite 59
Beispiel 18.1: Flüchtigen Zustand sichern (vor jeder Abschaltung) … Seite 62
Beispiel 18.2: Ein forensisches Abbild mit FTK Imager erstellen (Konzept) … Seite 62
Beispiel 18.3: Arbeitsspeicher sichern (Konzept) … Seite 62
Beispiel 19.1: Ablauf eines strukturierten AD-Audits (Konzept) … Seite 65
Beispiel 19.2: Ein Finding strukturiert dokumentieren … Seite 65
Beispiel 20.1: Grundgliederung eines Sicherheitsberichts … Seite 68
Beispiel 20.2: Eine Management Summary (Auszug, Konzept) … Seite 68

Index

AMSI – 50, 52
Angriffspfad – 17, 19, 32, 33, 34, 70
BloodHound – 17, 19, 32, 33, 34, 70
Chain of Custody – 62, 64
CVSS – 65, 66, 67, 70
Kerberoasting – 35, 36, 37, 70
Lateral Movement – 14, 15, 16, 56
LLMNR – 38, 39, 40
Management Summary – 68, 69, 70
Meterpreter – 47, 48, 49
NTDS.dit – 41, 43
NTLM-Relay – 38, 39, 40
Obfuskation – 50, 52
Order of Volatility – 62, 64
Pass-the-Hash – 41, 43
Privilege Escalation – 10
SAM – 12, 15, 16, 17, 18, 19, 20, 21
SharpHound – 32, 33, 34
Sysinternals – 26, 27
WinPEAS – 29, 30, 31